Understanding TypeScript

Section 1: Getting Started

Welcome to the Course!

1. Superset to JS
    1. Builds on JS
2. Less error-prone, better, cleaner code
3. New features in JS
4. Learn TS from scratch
    1. Basics
    2. How it works
    3. Where to use it
    4. Different advantages it offers
    5. Core types it provides
    6. Advanced typescript features
5. Theory + real projects
    1. With React
    2. With Node.js & Express
6. Why & How
7. With or without framework

What is TypeScript & Why Should You Use It?

1. What is TypeScript?
    1. A JavaScript superset
        1. A language building up on JavaScript
            1. Takes JS & adds new features + advantages
                1. Makes it easier & powerful
    2. Disadvantages
        1. Cannot be executed directly on browser or Node.js
    3. Role:
        1. It is a tool
            1. Powerful compiler compiles typescript code into JS (result)
                1. We get normal JS at the end
        2. How does new features of TS produce JS?
            1. Features are compiled to JS "workarounds", possible errors are thrown
                1. Nicer syntax & easier to write will be compiled to more complex JS snippet (would have been written by hand otherwise)
        3. It adds types
            1. Feature to JS which will give us an opportunity to identify errors in the code before it runs and errors occur at runtime
                1. Extra error checking to catch and fix errors during development
2. Why TypeScript?
    1. Example:

```
function add(num1, num2) {
  return num1 + num2;
}

console.log(add("2", "3"));
```

        1. JS will concatenate instead of adding numbers
            1. No runtime error but logical mistake
                1. Huge problems
            2. Mitigation strategies
                1. Add if check to add function
                2. Validate & sanitize user input at runtime
                    1. Instead if we can catch errors like this during development
                        1. TypeScript - Developers can write better code and avoid such errors

Join Our Learning Community

1. https://academind.com/community/

Installing & Using TypeScript

1. JS function

```
function add(num1, num2) {
  if (typeof num1 === "number" && typeof num2 === "number") {
    return num1 + num2;
  } else {
    return +num1 + +num2;
  }
}
```

    1. We don't want the extra code to check but we want to avoid passing non numbers
2. Typescript
    1. typescriptlang.org

```
npm install -g typescript # **(M)**
```

```
tsc <file>.ts
```

3. Code: using-ts.ts

```
const button = document.querySelector("button")!;
const input1 = document.getElementById("num1")! as HTMLInputElement; // never yields null
const input2 = document.getElementById("num2")! as HTMLInputElement; // never yields null

function add(num1: number, num2: number) {
  return num1 + num2;
}

button.addEventListener("click", function () {
  console.log(add(+input1.value, +input2.value));
});
```

    1. JS:

```
var button = document.querySelector("button");
var input1 = document.getElementById("num1"); // never yields null
var input2 = document.getElementById("num2"); // never yields null
function add(num1, num2) {
    return num1 + num2;
}
button.addEventListener("click", function () {
    console.log(add(+input1.value, +input2.value));
});
```

TypeScript Advantages - Overview

1. It makes writing clean code easier
2. It adds types
    1. To avoid unexpected and unnecessary errors by using types
    2. IDEs have support for TS
        1. Errors
        2. Autocomplete
3. Next-gen JS features (compiled down for older browsers)
    1. They get compiled to JS workarounds that even work in older browsers
        1. It is like Babel
            1. Already built into TSC
4. Non-JavaScript features like interfaces or generics
    1. Only TS understands them
        1. They help us give clearer errors
5. Meta-programming features like decorators
    1. More later
6. Rich configuration Options
    1. More later
        1. Compiler and how to configure it
            1. We can make it stricter or loser
            2. To make it behave like the way we want it to behave
            3. Tools like VS code has support for JS with TS under the hood (without explicitly using TS)

Course Outline

1. Getting Started
2. TypeScript Basics
    1. Core types
    2. How it generally works
    3. Some new features
    4. All needed for good understanding of TS
3. Compiler & Configuration Deep Dive
    1. What can be configured
    2. What the settings do
4. Working with Next-gen JS Code
    1. How do they work
    2. How to use them in TS
5. Classes & Interfaces
    1. In vanilla JS (classes)
    2. What they are
    3. What they do
    4. Why use them
6. Advanced Types & TypeScript Features
    1. Build on the basics
7. Generics
    1. What is it
    2. Why is it helpful
8. Decorators
    1. Cool feature
    2. Build useful decorators
9. Time to practice - full project
    1. With TS from scratch
        1. We see features in action
        2. How they work together
        3. How they simplify the pros
10. Working with Namespaces & Modules
    1. Makes code more readable and manageable
11. Webpack & TypeScript
    1. Buildtool in modern front-end web-development
        1. We can us with TS to get project better setup and make things easier
            1. Great dev experience

        2. Code that works well for end-users
    12. Third-party libraries & TypeScript
        1. Some libraries embraced TS but others did not
            1. How to work with both types of libraries
    13. React + TypeScript & NodeJS + TypeScript

How to Get the Most Our of the Course

1. Watch the Videos!
    1. Go through step-by-step
    2. All modules and lectures build up on each other
    3. Skip certain modules if you already know
    4. At your speed!
2. Code Along
    1. We learn better if we write code on our own and find and fix mistakes
        1. Pause & Rewind
3. Practice
    1. Real projects
    2. Small demos
    3. Pause and do next steps
    4. Advance on your own (partly)
4. Debug & Search
    1. Use attached Code, Google, Udemy Search
5. Ask & Answer
    1. Ask in Q&A section, but also help others!

Setting up a Code Editor / IDE

1. Code editor
    1. VS Code - good TS support (built in)
        1. Extensions
            1. ESLint - code quality checks
            2. Material Icon Theme
            3. Path Intellisense - better support for imports
            4. Prettier
            5. TSLint - merged with ESLint
        2. Settings
            1. Fine tune

Course Setup

1. Alternatives:
    1. IDEs offered by Jetbrains
        1. Webstorm
        2. Get a Jetbrains IDE for free from 6 months
            1. Enter coupon code: MAX_JETBRAINS
                1. Usage: https://sales.jetbrains.com/hc/en-gb/articles/206544449-I-received-a-coupon-code-for-a-JetBrains-license-how-can-I-use-it

The Course Project Setup

1. Starter project:

```
npm init
npm i --save-dev lite-server

// package.json

"start": "lite-server"

npm start
```

Section 2: TypeScript Basics & Basic Types

Module Introduction

1. Types offered and supported
    1. How to assign types
    2. How to reap the benefits of using types

Using Types

1. Types in TS
    1. TS adds many more types than what JS has
    2. TS enables us to write our own types
2. Core Types
    1. `number` - common number type (no ints, floats, ...)
        1. 1, 5.3, -10
        2. All are numbers and no differentiation between integers or floats
    2. `string` - Text
        1. 'Hi', "Hi", `Hi` (JS and TS) - template literals for dynamic injection of data
        2. All text values
    3. `boolean` - true, false
        1. Just these two, no "truthy" or "falsy" values (as in JS)

3. Typescript's type system only helps you during development (i.e. before the code gets compiled) - It doesn't change how JS works
    1. Adds extra sanity checks
        1. It helps during development but does not change runtime code
4. Code: app.ts

```
function add(n1: number, n2: number) {
  return n1 + n2;
}

const number1 = 5;
const number2 = 2.8;

const result = add(number1, number2);
console.log(result);
```

    1. TypeScript support is available only during development and not during runtime

TypeScript Types vs JavaScript Types

1. Code: app.ts

```
function add(n1: number, n2: number) {
  // console.log(typeof number1);
  if (typeof n1 !== "number" || typeof n2 !== "number") {
    // We are checking something which could have been avoided during development.
    // We could have prevented it using TS
    // If we use TS to call the function, we can throw an error during development
    // Helps fix bugs earlier
    throw new Error("Incorrect input!");
  }
  return n1 + n2;
}

const number1 = 5; // same as 5.0
const number2 = 2.8;

const result = add(number1, number2);
console.log(result);
```

Important: Type Casing

1. Core primitive types in TS are lowercase: `string`, `number` etc.

Working with Numbers, Strings & Booleans

1. Code:

```
function add(
  n1: number,
  n2: number,
  showResult: boolean,
  resultPhrase: string
) {
  const result = n1 + n2;
  if (showResult) {
    console.log(resultPhrase + result);
  } else {
    return result;
  }
}

const number1 = 5; // same as 5.0
const number2 = 2.8;
const printResult = true;
const resultPhrase = "Result is: ";

add(number1, number2, printResult, resultPhrase);
```

Type Assignment & Type Inference

1. TS does its best to understand which type we have in a variable or constant
2. Code:

```
let number1 = 5; // same as let number1: number = 5; // not a good practice
```

    1. If we are not initializing, then it is good to give the type

```
let number1: number; // otherwise TS allows any type
```

    2. Throws error if the following is done:

```
let resultPhrase = "Result is: ";
resultPhrase = 5;
```

Quiz 1: Understanding Types

Object Types

1. `object` type

```
{
    age: 30
```

```
}
```

1. Any JS object + more specific types (type of object) are possible (there are more specific versions of objects in TS - object that have properties, constructor function etc.)
2. Code:

```
console.log(person.nickname); // throws error

// Object type infererred by typescript
const person: {
    name: string;
    age: number;
}
```

    1. `object` - generic object type

```
person: object = {
    // ...
};

person.name; // fails
```

    2. Explicit declaration

```
const person: {
  name: string;
  age: number;
} = {
  name: "Maximilian",
  age: 30,
};

console.log(person);
console.log(person.age);
```

        1. It is only a TS representation and does not produce any additional JS code

    3. Object type for nested objects:

```
const product = {
    id: 'abc1',
    price: 12.99,
    tags: ['great-offer', 'hot-and-new'],
    details: {
        title: 'Red Carpet',
        description: 'A great carpet - almost brand-new!'
    }
};

// Type
const prouct: {
    id: string;
    price: number;
    tags: string[];
    details: {
        title: string;
        description: string;
    }
}
```

Nested Objects & Types

Arrays Types

1. [1, 2, 3]
    1. Any JavaScript array, type can be flexible or strict (regarding the element types)
2. Code:

```
const person = {
  name: "Maximilian",
  age: 30,
  hobbies: ["Sports", "Cooking"],
};

// let favoriteActivities: string[];
// favoriteActivities = ["Sports", 1];

let favoriteActivities: any[]; // any datatype - not recommended
favoriteActivities = ["Sports", 1];

console.log(person);
console.log(person.age);

for (const hobby of person.hobbies) {
  console.log(hobby.toUpperCase()); // works because hobby is identified as string
  // console.log(hobby.map()); // error because hobby is not an array
}
```

Working with Tuples

1. Tuple - JS doesn't have it
    1. [1, 2] - Added by TypeScript: Fixed-length array (and fixed types)
        1. Array example:

```
const person = {
  name: "Maximilian",
  age: 30,
  hobbies: ["Sports", "Cooking"],
  role: [2, "author"],
};

person.role.push("admin"); // works
person.role[1] = 10; // works

person.role.push('admin'); // allowed because TS unfortunately cannot catch this error

person.role = [0, 'admin']; // allowed
```

1. Tuples help us maintain even more strictness

2. Code:

```
const person: {
  name: string;
  age: number;
  hobbies: string[];
  role: [number, string]; // tuple. fixed types, fixed size
} = {
  name: "Maximilian",
  age: 30,
  hobbies: ["Sports", "Cooking"],
  role: [2, "author"],
};

// person.role.push("admin"); // works
// person.role[1] = 10; // works

// let favoriteActivities: string[];
// favoriteActivities = ["Sports", 1];

let favoriteActivities: any[]; // any datatype - not recommended
favoriteActivities = ["Sports", 1];

console.log(person);
console.log(person.age);

for (const hobby of person.hobbies) {
  console.log(hobby.toUpperCase()); // works because hobby is identified as string
  // console.log(hobby.map()); // error because hobby is not an array
}
```

Working with Enums

1. Enum - JS doesn't know it
    1. `enum { NEW, OLD }`
        1. Added by TS: Automatically enumerated global constant identifiers

2. Code:

```
enum Role { // we can assign number, text, ...
  ADMIN = 5,
  READ_ONLY,
  AUTHOR,
}

const person = {
  name: "Maximilian",
  age: 30,
  hobbies: ["Sports", "Cooking"],
  role: Role.AUTHOR,
};

let favoriteActivities: any[]; // any datatype - not recommended
favoriteActivities = ["Sports", 1];

console.log(person);
console.log(person.age);

for (const hobby of person.hobbies) {
  console.log(hobby.toUpperCase()); // works because hobby is identified as string
  // console.log(hobby.map()); // error because hobby is not an array
}

if (person.role === Role.AUTHOR) {
  console.log("is author");
}
```

The "any" Type

1. Any - *
    1. Any kind of value, no specific type assignment
        1. TS allows any type
    2. We want to avoid it
        1. It takes away the advantages given by TS
            1. It ensures TS doesn't check anything
                1. It can be used as fallback
    3. If we know the type, etter to be specific

Union Types

1. Code: We might need a runtime type check

```
function combine(input1: number | string, input2: number | string) {
  let result;
  if (typeof input1 === "number" && typeof input2 === "number") {
    result = input1 + input2;
  } else {
    result = input1.toString() + input2.toString();
  }
  return result;
}

const combinedAges = combine(30, 26);
console.log(combinedAges);

const combinedNames = combine("Max", "Anna");
console.log(combinedNames);
```

Literal Types

1. Example:

```
const number2 = 2.8; // infers number as the type because it is const
```

2. Code:

```
function combine(
  input1: number | string,
  input2: number | string,
  resultConversion: "as-number" | "as-text"
) {
  let result;
  if (typeof input1 === "number" && typeof input2 === "number") {
    result = input1 + input2;
  } else {
    result = input1.toString() + input2.toString();
  }

  if (resultConversion === "as-number") {
    return +result;
  } else {
    return result.toString();
  }
}

const combinedAges = combine(30, 26, "as-number");
console.log(combinedAges);

const combinedStringAges = combine("30", "26", "as-number");
console.log(combinedStringAges);

const combinedNames = combine("Max", "Anna", "as-text");
console.log(combinedNames);
```

Type Aliases / Custom Types

1. New type that stores union type
    1. Solution: Alias
2. Code:

```
type Combinable = number | string; // Any name not built into JS or TS
type ConversionDescriptor = "as-number" | "as-text";

function combine(
  input1: Combinable,
  input2: Combinable,
  resultConversion: ConversionDescriptor
) {
  let result;
  if (typeof input1 === "number" && typeof input2 === "number") {
    result = input1 + input2;
  } else {
    result = input1.toString() + input2.toString();
  }

  if (resultConversion === "as-number") {
    return +result;
  } else {
    return result.toString();
  }
}

const combinedAges = combine(30, 26, "as-number");
console.log(combinedAges);

const combinedStringAges = combine("30", "26", "as-number");
console.log(combinedStringAges);

const combinedNames = combine("Max", "Anna", "as-text");
console.log(combinedNames);
```

Type Aliases & Object Types

1. We can use aliases for complex types too

```
type User = { name: string; age: number };
const u1: User = { name: 'Max', age: 30 }; // this works!
```

    1. Example: Simplification

```
function gree(user: { name: string; age: number }) {
    console.log('Hi, I am ' + user.name);
}

function isOlder(user: { name: string; age: number },checkAge: number) {
    return checkAge > user.age;
}

// Simplified code
type User = { name: string; age: number };

function greet(user: User) {
    console.log('Hi, I am ' + user.name);
}

function isOlder(user: User, checkAge: number) {
    return checkAge > user.age;
}
```

Quiz 2: Core Types & Concepts

Function Return Types & "void"

2. Code: app.ts

```
function printResult(num: number): void {
  // function cannot return `undefined`
  console.log("Result: " + num);
} // TS can infer the return type. JS doesn't understand it

// ...

function printResult(num: number): undefined {
  // function can return `undefined` if `return` is the last statement
  console.log("Result: " + num);
  return;
} // TS can infer the return type. JS doesn't understand it
```

    1. function returning `undefined` is rare

Functions as Types

1. Code:

```
let combineValues: (a: number, b: number) => number; // parameters can be anything

// combineValues = printResult; // throws error
combineValues = add;

console.log(combineValues(8, 8));

// combineValues = 5; // error
```

Function Types & Callbacks

1. Code:

```
function addAndHandle(n1: number, n2: number, cb: (num: number) => void) {
  // void says we will ignore anything returned by callback
  const result = n1 + n2;
  cb(result);
}

addAndHandle(10, 20, (result) => {
  // type is inferred
  console.log(result);
});
```

Quiz 3: Functions & Types

The "unknown" Type

1. Code:

```
let userInput: unknown; // we don't know yet.
let userName: string;

userInput = 5; // allowed
userInput = "Max"; // allowed
// userName = userInput; // error - unknown cannot assigned to a string. If `any` is used, it works. Unknown is more restrictive
```

The "never" Type

1. Code:

```
function generateError(message: string, code: number): never {
  // the function never returns. The flow is interrupted at the throw. It crashes the script
  throw { message: message, errorCode: code };
}

generateError("An error occurred!", 500);

// Another example:
function generateLoop(): never {
  while (true);
}
```

Wrap Up

Useful Resources & Links

1. https://www.typescriptlang.org/docs/handbook/2/everyday-types.html
2. Resources

Section 3: The TypeScript Compiler (and its Configuration)

Module Introduction

Using "Watch Mode"

1. Watch mode:

```
tsc app.ts --watch
tsc app.ts -w
```

Compiling the Entire Project / Multiple Files

1. Multiple ts files
    1. Tell TSC that the directory is a project

    ```
    tsc --init
    ```

        1. `tsconfig.json`
            1. Folder and subfolders will be managed by TS
    2. Compiling all TS files in the project

    ```
    tsc
    ```

    ```
    tsc -w # for all files
    ```

Including & Excluding Files

1. Importing using modules
2. tsconfig.json

```
"exclude": ["analytics.ts", "*.dev.ts", "**/*.test.ts", "node_modules"], // `node_modules` would be the default
"include": ["app.ts", "analytics.ts"], // compiles only the files listed here. Compilation: include - exclude
"files": ["app.ts"] // only individual files
```

Setting a Compilation Target

1. tsconfig.json

```
"target": "es2016"
```

    1. More modern version we pick, the more concise the generated code is
    2. TS has to compile less code
    3. TS has to workaround non-existing features in lesser situations

Understanding TypeScript Core Libs

1. `// "lib": []` - assumes defaults
    1. Which default objects and features TS knows
        1. Working with the DOM, say (TS will know DOM objects and features)
    2. Example:

    ```
    const button = document.querySelector("button")!;

    button?.addEventListener("click", () => {
      console.log("Clicked!");
    });
    ```

        1. Default depends on JS target
            1. ES6 - Includes all features globally available for ES6
            2. All DOM APIs are available
        2. `"lib": []`
            1. No defaults are assumed
        3. `"lib": ["DOM"]`
    3. Default:

    ```
    "lib": [
          "DOM",
          "ES2015",
          "DOM.Iterable",
          "ScriptHost"
        ] /* Specify a set of bundled library declaration files that describe the target runtime environment. */,
    ```

2. `"allowJs": true`
    1. Allows JS files to be included in the compilation

    2. Compiles JS files

3. `"checkJs": true`

    1. Only checks for syntax issues and reports
    2. Usecase: We want to check vanella JS files in the project

More Configuration & Compilation Options

1. `"jsx": "preserve"`
2. `"declaration": true`
    1. `.d.ts` - for libraries
        1. For manifest files which describes all the types in the project
3. `"declarationMap": true`

Working with Source Maps

1. Dev tools
    1. If we have more complex TS code
        1. It would be nice to have TS files in Dev tools for debugging
            1. Solution: `"sourceMap": true`
                1. `app.js.map` generated
                    1. They act as bridge understood by browsers to connect JS files to input files
                        1. Browser shows TS files along with JS files
                            1. We can place breakpoints and browser pauses at the breakpoints

rootDir and outDir

1. `"outDir": "./"`
    1. To specify the folder which has output
        1. `"./dist"`
        2. `<script src="dist/app.js" defer></script>`
2. `"rootDir": "./"`
    1. `"./src"`
        1. Maintains the same project structure in the `dist` folder as in the `src` folder
3. `"removeComments": true` - removes all comments in JS
    1. Makes output files smaller
4. `"noEmit": true` - doesn't generate JS files, but used to check if input files are correct without generating the output files
    1. For errors
5. `"downlevelIteration": true` - Outputs exact compilation. Works in niche cases. It will output more verbose code
    1. Turn it on if there are loops and if generated code behaves differently than it should.

Stop Emitting Files on Compilation Errors

1. `"noEmitOnError": true`
    1. Nothing is generated if any file has errors

Strict Compilation

1. `"strict": true`

    1. Strict type-checking

        1. The same as setting the following options:

```
"noImplicitAny": true,                          /* Enable error reporting for expressions and declarations with an implied 'any' type
"strictNullChecks": true,                       /* When type checking, take into account 'null' and 'undefined'. */
"strictFunctionTypes": true,            /* When assigning functions, check to ensure parameters and the return values are subtype-com
"strictBindCallApply": true,            /* Check that the arguments for 'bind', 'call', and 'apply' methods match the original functi
"strictPropertyInitialization": true,   /* Check for class properties that are declared but not set in the constructor. */
"strictBuiltinIteratorReturn": true,    /* Built-in iterators are instantiated with a 'TReturn' type of 'undefined' instead of 'any'.
"noImplicitThis": true,                         /* Enable error reporting when 'this' is given the type 'any'. */
```

Code Quality Options

1. `"noImplicitAny": true`

    1. Example:

```
function sendAnalytics(data) {
  console.log(data);
}

sendAnalytics("The data");
```

        1. Compiler complains that `data` implicitly has `any` type
            1. If `false`, the error goes away
        2. It ensures that we are clear about our parameters and values we are working with in our code

2. TS can infer types:

```
console.log("Sending...");
let logged;

function sendAnalytics(data: string) { // here the function is defined before we call it, so TS doesn't know what type of value is going to be p
  console.log(data);
  logged = true;
  console.log(logged); // TS interprets it as `boolean`
}
```

```
sendAnalytics("The data");
```

3. `strictNullCheck`
    1. Example:

```
const button = document.querySelector("button")!;

button.addEventListener("click", () => {
  console.log("Clicked!");
});
```

        1. A cleaner solution:

```
const button = document.querySelector("button");

button?.addEventListener("click", () => {
  console.log("Clicked!");
});
```

4. `"strictFunctionTypes": true`
    1. For catching niche bugs (we may not have them in many situations)
    2. Related to function types - if we define how a function should look like regarding params and return values, then we can introduce bugs if we work with classes and inheritance
5. `"strictBindCallApply": true`
    1. Example:

```
const button = document.querySelector("button"); //

function clickHandler(message: string) {
  console.log("Clicked " + message);
}

button?.addEventListener("click", clickHandler.bind(null, "You're welcome!")); // checks if what we are binding makes sense
```

        1. Ensures that `bind` or `apply` is defined in a way that doesn't work in the code
6. `"noImplicitThis": true`
    1. Warns if we use `this` keyword in a place that makes it unclear what `this` refers to
7. `"alwaysStrict": true`
    1. Ensures that JS files always have `"use strict";`

Code Quality Options

1. Options:

```
"noUnusedLocals": true,
"noUnusedParameters": true,
"noImplicitReturns": true,
"noFallthroughCasesInSwitch": true,
```

    1. `"noImplicitReturns": true` - If some of the branches return but some do not have return statement
        1. We either return in all cases or don't return anything
2. `/* Source Map Options */`
    1. Used to tweak source maps
        1. Default settings are fine
3. `experimentalDecorators` - allows us to use some experimental features. They are not sure if it will end up in JS sometime in the future

Debugging with Visual Studio Code

1. How to debug and have a great development flow
    1. ESLint - to improve code quality
    2. Prettier - automatically format code
    3. Debugger for chrome
        1. To debug files even from inside VS Code
        2. JavaScript Debugger - now
            1. Needs enabling `sourceMap`
            2. Go to Debug
                1. Start debugging (in VS Code)
        3. launch.json

```
"url": "http://localhost:3000",
```

        4. Debugging view
            1. Inspect variables
            2. Watch

3. Call stack

Wrap Up

Useful Resources & Links

1. tsconfig
2. compiler config docs
3. VS Code TS Debugging

Section 4: Next-generation JavaScript & TypeScript

Module Introduction

1. Modern features that may not run on older browsers

"let" and "const"

2. Code:

```
const userName = "Max";
// userName = ""; // now allowed
let age = 20;

age = 30;

// var a = 10;
// let - scope in which a variable is available
// var - global and function scopes. Variables defined inside a function are available in the function.

var result;

function add(a: number, b: number) {
  var result;
  result = a + b;
  return result;
}

console.log(result);

// difference. var has only global and function scope. a variable defined inside `if` will get promoted to global scope
// let - available inside a block-scope (or lower block) only or global scope
```

Arrow Functions

1. Code:

```
const add = (a: number, b: number) => a + b;

console.log(add(2, 5));

const printOutput: (a: number | string) => void = (output) =>
  console.log(output);

const button = document.querySelector("button");

button?.addEventListener("click", (event) => console.log(event)); // concise syntax is allowed in this case, because we are not defining the fun
```

Default Function Parameters

1. Code:

```
const add = (a: number, b: number = 1) => a + b;

console.log(add(2, 5));
console.log(add(1));
```

The Spread Operator (...)

1. Code:

```
const hobbies = ["Sports", "Cooking"];
const activeHobbies = ["Hiking", ...hobbies];

// activeHobbies.push(hobbies[0], hobbies[1]);
// activeHobbies.push(...hobbies);

const person = {
  name: "Max",
  age: 30,
};

// const copiedPerson = person;
const copiedPerson = { ...person };
```

Rest Parameters

1. Code:

```
const add = (...numbers: number[]) =>
  numbers.reduce((sum, item) => sum + item, 0);
```

```
const addedNumbers = add(5, 10, 2, 3.7);
console.log(addedNumbers);
```

Array & Object Destructuring

1. Code:

```
const [hobby1, hobby2, ...remainingHobbies] = hobbies;
const { firstName: userName, age } = person; // order is not important. By keys
console.log(userName, age);
```

How Code Gets Compiled & Wrap Up

1. `"target": "es5"`
    1. Compiles TS code to ES5 code

Useful Resources & Links

Section 5: Classes & Interfaces

Module Introduction

1. Classes & Interfaces
    1. Modern vanella JS has classes
2. Module content
    1. What & Why?
    2. Classes & Inheritance
    3. Interfaces
        1. How are they compiled to JS

What are Classes?

1. What's Object-Oriented Programming (OOP)?
    1. Work with (real-life) entities in your code
        1. Objects that resemble real-life objects
            1. Makes it easy for developers to work
                1. ProductList
                    1. Renders a list of roducts which were fetched from a server (database)
                    2. Object holds rendering + fetching logic
                2. Product
                    1. Renders details about a product and allows addition to cart
                    2. Object holds rendering + cart-adding logic
                3. ShoppingCart
                    1. Renders cart products and total and allows user to order them
                    2. Object holds rendering + ordering (server communication) logic
2. Classes & Interfaces
    1. We can split our application into parts managed by objects
        1. Classes
    2. Objects
        1. The things we work with in code
            1. Data structures to store data and methods to execute on the data structures
        2. Instances of classes (= based on classes)
        3. Class-based creation is an alternative to using object literals!
    3. Classes
        1. Blueprints for objects (theoretical definition)
            1. How objects should look like
                1. What data & methods they should have
        2. Define how objects look like, which properties and methods they have
        3. Classes make creation of multiple, similar objects much easier

Constructing a First Class

1. Company internal tool
    1. Departments
2. Code:

```
class Department {
  firstName: string;

  constructor(n: string) {
    this.firstName = n;
  }
}

const accounting = new Department("Accounting");
console.log(accounting);
```

Compiling to JavaScript

Constructor Functions & The "this" Keyword

1. Code:

```
class Department {
  departmentName: string;
```

```
  constructor(n: string) {
    this.departmentName = n;
  }

  describe() {
    console.log("Department: " + this.departmentName);
  }
}

const accounting = new Department("Accounting");
console.log(accounting);

accounting.describe();

const accountingCopy = { describe: accounting.describe };
accountingCopy.describe(); // pointer to `describe` method but `undefined`. `this` refers to the calling object
```

2. Code:

```
class Department {
  departmentName: string;

  constructor(n: string) {
    this.departmentName = n;
  }

  describe(this: Department) {
    // used to avoid undefined
    // an instance of Department class
    console.log("Department: " + this.departmentName);
  }
}

const accounting = new Department("Accounting");
console.log(accounting);

accounting.describe();

const accountingCopy = { describe: accounting.describe };
// accountingCopy.describe(); // doesn't refer to object of type Department
```

"private" and "public" Access Modifiers

1. Vanilla JS private field syntax doesn't use `private` and `public` keywords
2. Code:

```
class Department {
  private departmentName: string;
  private employees: string[] = []; // `public` is default. JS now knows.

  constructor(n: string) {
    this.departmentName = n;
  }

  describe(this: Department) {
    // used to avoid undefined
    // an instance of Department class
    console.log("Department: " + this.departmentName);
  }

  addEmployee(employee: string) {
    // validation
    this.employees.push(employee);
  }

  printEmployeeInformation() {
    console.log(this.employees.length);
    console.log(this.employees);
  }
}

const accounting = new Department("Accounting");
console.log(accounting);

accounting.describe();

const accountingCopy = { describe: accounting.describe };
// accountingCopy.describe(); // doesn't refer to object of type Department

accounting.addEmployee("Max");
accounting.addEmployee("Manu");
accounting.printEmployeeInformation();
```

Shorthand Initialization

1. Code:

```
class Department {
  constructor(
    private departmentName: string,
    private employees: string[] = []
  ) {}

  describe(this: Department) {
    // used to avoid undefined
    // an instance of Department class
    console.log("Department: " + this.departmentName);
  }
```

```
    addEmployee(employee: string) {
      // validation
      this.employees.push(employee);
    }

    printEmployeeInformation() {
      console.log(this.employees.length);
      console.log(this.employees);
    }
  }
```

"readonly" Properties

1. Code:

```
class Department {
  constructor(
    private readonly id: string,
    private departmentName: string,
    private employees: string[] = []
  ) {}
```

    1. We cannot reassign a value to the field later.

Quiz 4: Class Basics

Inheritance

1. Code:

```
class ITDepartment extends Department {
  // We can inherit only from one class
  constructor(id: string, public admins: string[]) {
    super(id, "IT"); // calls the constructor of the base class. Called first before doing anything with `this` keyword
  }
}

class AccountingDepartment extends Department {
  constructor(id: string, private reports: string[]) {
    super(id, "Accounting");
  }

  addReport(text: string) {
    this.reports.push(text);
  }

  printReports() {
    console.log(this.reports);
  }
}

const it = new ITDepartment("345dfd", ["Max", "Duke"]);
console.log(it);

const accountingDep = new AccountingDepartment("d3", []);
accountingDep.addReport("Something went wrong...");
accountingDep.printReports();
```

Overriding Properties & The "protected" Modifier

1. Code:

```
class Department {
  constructor(
    // ...
    protected employees: string[] = [] /* It is available in this class and all classes that inherit this class */
  ) {}
  // ...
}

class AccountingDepartment extends Department {
  // ...

  addEmployee(name: string) {
    if (name === "Max") {
      return;
    }
    this.employees.push(name); // if it is private. They are only accessible in which they are defined.
  }

  // ...
}
```

Getters & Setters

1. Code:

```
class AccountingDepartment extends Department {
  private lastReport: string;

  get mostRecentReport() {
    if (this.lastReport) {
      return this.lastReport;
    }
    throw new Error("No report found.");
  }

  set mostRecentReport(value: string) {
    if (!value) {
```

```
      throw new Error("Please pass in a valid value");
    }
    this.addReport(value);
  }
  // ...
}

console.log(accountingDep.mostRecentReport); // no parans are required
// accountingDep.mostRecentReport = ""; // throws an error
accountingDep.mostRecentReport = "Year-end report";
```

Static Methods & Properties

1. Properties and methods added to classes (not instance of the class)
    1. Utility functions
    2. Global constants
2. We cannot access static member using `this` inside the class.
3. Code:

```
class Department {
  static fiscalYear = 2020;

  constructor(
    private readonly id: string,
    private departmentName: string,
    protected employees: string[] = [] /* It is available in this class and all classes that inherit this class */
  ) {}

  static defineEmployee(name: string) {
    return { name: name };
  }
  // ...
}

console.log(Math.PI);
console.log(Math.pow(2, 10));

const employee1 = Department.defineEmployee("Max");
console.log(employee1);
console.log(Department.fiscalYear);
```

Abstract Classes

1. Abstract methods:
    1. Use-cases:
        1. To ensure a method is available in all subclasses
        2. When we cannot provide a generic implementation but the implementation depends on the subclasses
2. We cannot instantiate an abstract class
3. Code:

```
abstract class Department {

  // ...

  abstract describe(): void;

  // ...
}

class ITDepartment extends Department {

  // ...

  describe(): void {
    console.log("IT Department - ID: " + this.id);
  }
}
```

Singletons & Private Constructors

1. Singleton
    1. Ensures that we have only one instance of the class
        1. We don't want multiple objects
2. Code:

```
class Singleton {
  private static instance: Singleton;

  private constructor() {}

  static getInstance() {
    if (this.instance) {
      // `this` refers to the class here
      return this.instance;
    }
    this.instance = new Singleton();
    return this.instance;
  }
}
```

```
const singleton1 = Singleton.getInstance();
const singleton2 = Singleton.getInstance();
```

Classes - A Summary

Quiz 5: Classes

A First Interface

1. Code:

```
interface Person {
  name: string;
  age: number;

  greet(phrase: string): void;
}

let user1: Person; // interface as a type. It is used to typecheck

user1 = {
  name: "Max",
  age: 30,
  greet(phrase: string): void {
    console.log(phrase + " " + this.name);
  },
};

user1.greet("Hi there - I am");
```

Using Interfaces with Classes

1. `type` and `interface`
    1. Interfaces can be used to define structured of objects
        1. It is clearer that it will define object
        2. It can be implemented by a class
            1. As a contract
    2. Types for objects other types like union, etc.
2. Code:

```
interface Greetable {
  name: string;

  greet(phrase: string): void;
}

class Person implements Greetable {
  constructor(public name: string) {}

  greet(phrase: string): void {
    console.log(phrase + " " + this.name);
  }
}

let user1: Greetable; // interface as a type. It is used to typecheck

user1 = new Person("Max");
user1.greet("Hi there - I am");
```

Why Interfaces?

1. We want to ensure that a class has the methods and properties
    1. Each class adds its own implementation
        1. Useful if we have other parts of code which relies on the structure
            1. The other code can use the class that implements the interface

Readonly Interface Properties

1. `readonly` modifier
    1. The value can be set only once
2. `public`, `private`, `protected` cannot be added
3. Code:

```
interface Greetable {
  readonly name: string;

  // ...
}
```

Extending Interfaces

1. Why split interfaces
    1. In some cases we can use some of them and not all
    2. We can `extends` multiple interfaces

Interfaces as Function Types

1. Code:

```
type AddFn = (a: number, b: number) => number;

let add: AddFn;

add = (n1: number, n2: number) => {
  return n1 + n2;
};

// Using interfaces for functions
interface AddFn2 {
  (a: number, b: number): number;
}

let add2: AddFn2;

add2 = (n1: number, n2: number) => {
  return n1 + n2;
};
console.log(add2(324, 433));
```

Optional Parameters & Properties

1. Optional using ?
    1. Properties
    2. Methods
    3. Parameters
2. Example:

```
interface Named {
  readonly name?: string;
  outputName?: string; // property might exist
}

interface Greetable extends Named {
  greet(phrase: string): void;
}

class Person implements Greetable {
  constructor(public name?: string) {}

  greet(phrase: string): void {
    console.log(phrase + " " + this.name);
  }
}
```

Compiling Interfaces to JavaScript

1. No translations for interfaces (JS doesn't know)
    1. Only for development and compilation

Quiz 6: Interfaces

Wrap Up

Useful Resources & Links

1. JS Classes
2. TS Interfaces

Section 6: Advanced Types

Module Introduction

1. Advanced concepts
    1. Intersection types
    2. Type guards
    3. Discriminated unions
    4. Type casting (if TS cannot find out the type)
    5. Function overloads
        1. Flexible functions

Intersection Types

1. Code

```
type Admin = {
  name: string;
  privileges: string[];
};

type Employee = {
  name: string;
  startDate: Date;
};

type ElevatedEmployee = Admin & Employee;

const e1: ElevatedEmployee = {
  name: "Max",
```

```
  privileges: ["start-server"],
  startDate: new Date(),
};

type Combinable = string | number;
type Numeric = number | boolean;
type Universal = Combinable & Numeric; // Intersection type: `number`
type Universal2 = Combinable | Numeric; // Union type: `string | number | boolean`
```

More on Type Guards

1. Code:

```
function add(a: Combinable, b: Combinable) {
  if (typeof a === "string" || typeof b === "string") {
    // typeguard. Ensures code runs properly at runtime
    return a.toString() + b.toString();
  }
  return a + b;
}

type UnknownEmployee = Employee | Admin;

function printEmployeeInformation(emp: UnknownEmployee) {
  console.log("Name: " + emp.name);
  if ("privileges" in emp) {
    console.log("Privileges: " + emp.privileges);
  }
}
printEmployeeInformation(e1);
printEmployeeInformation({ name: "Manu", startDate: new Date() });

class Car {
  drive() {
    console.log("Driving...");
  }
}

class Truck {
  drive() {
    console.log("Driving a truck...");
  }

  loadCargo(amount: number) {
    console.log("Loading cargo ..." + amount);
  }
}

type Vehicle = Car | Truck;

const v1 = new Car();
const v2 = new Truck();

function useVehicle(vehicle: Vehicle) {
  vehicle.drive();
  // if ("loadCargo" in vehicle) vehicle.loadCargo(1000);
  if (vehicle instanceof Truck) vehicle.loadCargo(1000); // Vanella JS. Cannot be used with interfaces because JS doesn't know interfaces. The c
}

useVehicle(v1);
useVehicle(v2);
```

Discriminated Unions

1. Code:

```
interface Bird {
    type: "bird"; // literal type
    flyingSpeed: number;
}

interface Horse {
    type: "horse";
    runningSpeed: number;
}

type Animal = Bird | Horse;

function moveAnimal(animal: Animal) {
    // if ("flyingSpeed" in animal)
    // console.log("Moving with speed: " + animal.flyingSpeed);

    let speed: number;
    switch (animal.type) {
        case "bird":
            speed = animal.flyingSpeed;
            break;
        case "horse":
            speed = animal.runningSpeed;
    }
    console.log("Moving at speed: " + speed);
}

moveAnimal({ type: "bird", flyingSpeed: 10 });
```

Type Casting

1. Code:

```
const userInputElement = <HTMLInputElement>(
  document.getElementById("user-output")
```

```
); // HTMLElement (generic type)

const userInputElement = document.getElementById(
  "user-output"
) as HTMLInputElement; // HTMLElement (generic type)

if (userInputElement) {
  (userInputElement as HTMLInputElement).value = "Hi there";
}
```

Index Properties

1. Objects that are flexible regarding the properties they might hold
2. Code:

```
interface ErrorContainer {
  // Which property names will be used and how many is not known in advance
  // { email: 'Not a valid email', username: 'Must start with a character!' }
  id: string; // is of the same type as below. We cannot have `number` type
  [prop: string]: string;
}

const errorBag: ErrorContainer = {
  id: "123adb",
  email: "Not a valid email",
  username: "Must start with a capital character!",
  1: "Not a valid number", // allowed because number can be interpreted as a key-type
};
```

Function Overloads

Code:

```
type Combinable = string | number;

// function add(n: number): number
function add(a: number, b: number): number;
function add(a: string, b: number): string;
function add(a: number, b: string): string;
function add(a: string, b: string): string;
function add(a: Combinable, b: Combinable) {
  if (typeof a === "string" || typeof b === "string") {
    return a.toString() + b.toString();
  }
  return a + b;
}

// const result = add(1, 5) as number;
// const result = add('Max', ' Schwarz') as string;

const result = add(1, "5"); // return: string
result.split("");
```

Optional Chaining

1. Code:

```
// Optional Chaining
const fetchedUserData = {
  id: "u1",
  name: "Max",
  job: { title: "CEO", description: "My own company" },
};

console.log(fetchedUserData?.job?.title);
```

Nullish Coalescing

1. Code:

```
// const userInput = null;
const userInput = "";

// const storedData = userInput || 'DEFAULT'; // if falsy. '' is also falsy
const storedData = userInput ?? "DEFAULT"; // `null` or `undefined` only
```

Quiz 7: Advanced Types

Wrap Up

Useful Resources & Links

1. More on advanced types

Section 7: Generics

Module Introduction

1. Exists in TS
    1. C#
    2. Java
2. Topics:
    1. What?

2. Generic Functions & Classes
3. Constraints
4. Special TypeScript Types

Build-in Generics & What are Generics?

1. Code:

```
// const names = ['Max', 'Manuel']; // string[]
// const names = []; // any[]
const names: Array<string> = []; // same as string[]
const names2: Array<string | number> = []; // same as (string | number)[];

names[0]?.split(" "); // allowed

const promise: Promise<string> = new Promise((resolve) => {
  setTimeout(() => {
    resolve("This is done!");
  }, 2000);
}); // Promise<unknown> otherwise

promise.then((data) => {
  data.split(" "); // allowed because data is defined to be `string`
});
```

Constructing a Generic Function

1. Code:

```
function merge<T extends Object, U extends Object>(objA: T, objB: U) {
  // returns intersection of types
  // return Object.assign(objA, objB);
  return Object.assign(objA, objB);
}

const mergeObj = merge({ name: "Max" }, { age: 20 });
console.log(mergeObj);
// mergeObj.name; // cannot be accessed. TS doesn't know it
// console.log((mergeObj as { name: string; age: number }).name);
console.log(mergeObj.name);
console.log(mergeObj.age);

const mergedObj2 = merge({ name: "Max", hobbies: ["Sports"] }, { age: 30 });
console.log(mergedObj2);

const mergedObj3 = merge<{ name: string }, { age: number }>(
  { name: "Max" },
  { age: 30 }
); // not required because TS infers types
console.log(mergedObj3);

const mergedObj4 = merge<string, number>("Max", 30);
console.log(mergedObj4);
```

Working with Constraints

1. Code:

```
function merge<T extends object, U extends object>(objA: T, objB: U) {
  // returns intersection of types
  // return Object.assign(objA, objB);
  return Object.assign(objA, objB);
}

const mergeObj = merge({ name: "Max" }, { age: 20 });
console.log(mergeObj);
// mergeObj.name; // cannot be accessed. TS doesn't know it
// console.log((mergeObj as { name: string; age: number }).name);
console.log(mergeObj.name);
console.log(mergeObj.age);

const mergedObj2 = merge({ name: "Max", hobbies: ["Sports"] }, { age: 30 });
console.log(mergedObj2);

const mergedObj3 = merge<{ name: string }, { age: number }>(
  { name: "Max" },
  { age: 30 }
); // not required because TS infers types
console.log(mergedObj3);

// const mergedObj4 = merge<string, number>("Max", 30); // fails silently otherwise because 'Max' and 30 are not objects and don't get merged
// console.log(mergedObj4);
```

Another Generic Function

1. Code: Elements that have `length` property

```
interface Lengthy {
  length: number;
}

function countAndDescribe<T extends Lengthy>(element: T): [T, string] {
  let descriptionText = "Got no value.";
  if (element.length > 0) {
    descriptionText = "Got " + element.length + " elements.";
  }
  d;
  return [element, descriptionText];
}
```

```
console.log(countAndDescribe("Hi there!"));
console.log(countAndDescribe(["Cookie", "Chocolate"]));
```

The "keyof" Constraint

1. Code: `keyof` ensures that the argument is a valid key

```
function extractAndConvert<T extends object, U extends keyof T>(
  obj: T,
  key: U
) {
  return "Value: " + obj[key];
}

extractAndConvert({ name: "Max" }, "name");
extractAndConvert({ name: "Max", age: 30 }, "age");
```

Generic Classes

1. Code: Generic code

```
// Generic classes
class DataStorage<T> {
  private data: T[] = [];

  addItem(item: T) {
    this.data.push(item);
  }

  removeItem(item: T) {
    if (this.data.indexOf(item) === -1) {
      return;
    }
    this.data.splice(this.data.indexOf(item), 1);
  }

  getItems() {
    return [...this.data];
  }
}

const textStorage = new DataStorage<string>();
textStorage.addItem("Max");
textStorage.addItem("Manu");
textStorage.removeItem("Max");
console.log(textStorage.getItems());

const numberStorage = new DataStorage<number>();

const objStorage = new DataStorage<object>();
objStorage.addItem({ name: "Max" });
objStorage.addItem({ name: "Manu" });
// ...
objStorage.removeItem({ name: "Manu" }); // keeps `Max` but it is a new object and not a reference to existing object
objStorage.removeItem({ name: "Max" }); // keeps `Max` but it is a new object and not a reference to existing object. It removes last element always.
console.log(objStorage.getItems());

const bobObj = { name: "Bob" };
objStorage.addItem(bobObj);
console.log(objStorage.getItems());
objStorage.removeItem(bobObj);
console.log(objStorage.getItems());
```

2. Code: Restricted to only primitive types

```
class DataStorage<T extends string | number | boolean> {
  private data: T[] = [];

  addItem(item: T) {
    this.data.push(item);
  }

  // removeItem<U>(item: T) // allowed
  removeItem(item: T) {
    if (this.data.indexOf(item) === -1) {
      return;
    }
    this.data.splice(this.data.indexOf(item), 1);
  }

  getItems() {
    return [...this.data];
  }
}

const textStorage = new DataStorage<string>();
textStorage.addItem("Max");
textStorage.addItem("Manu");
textStorage.removeItem("Max");
console.log(textStorage.getItems());

const numberStorage = new DataStorage<number>();
```

A First Summary

1. They give flexibility combined with type-safety

Generic Utility Types

1. Types exist in TS only - for extra type safety during compilation
2. Utilities take something and do something around it.
    1. Lock them down
    2. Make fields optional
    3. ...
3. Code:

```
function constructCourseGoal(title: string, description: string, date: Date) {
  // let courseGoal: CourseGoal = {}; // error
  let courseGoal: Partial<CourseGoal> = {}; // turns into a type (object type) and make all fields optional
  courseGoal.title = title;
  courseGoal.description = description;
  courseGoal.completeUntil = date;
  return courseGoal as CourseGoal; // At this time it is complete CourseGoal
}

const names: Readonly<string[]> = ["Max", "Anna"];
// names.push('Manu'); // throws an error because we cannot modify the array. Allowed for objects too
```

Generic Types vs Union Types

1. Union can cause mixing of types

```
class DataStorage {
  private data: (string | number | boolean)[];
}
```

2. With generics, we can use data of a particular type only wherever the generic type is used
    1. They lock in a type

Quiz 8: Generics

Useful Resources & Links

1. More on generics

Section 8: Decorators

Module Introduction

1. Useful for meta-programming
    1. Well suited for writing code that makes it useful for other developers (not users of the code)
        1. Example:
            1. To ensure a class is used correctly
            2. To ensure a method in a class is used correctly
            3. Hidden transformations
2. Topics:
    1. What?
    2. Decorator usage
    3. Examples!

A First Class Decorator

1. tsconfig.json

    ```
    "experimentalDecorators": true
    ```

2. Decorator: It is a function
    1. It is applied to something (class say) in a certain way
3. Code

```
function Logger(constructor: Function) {
  console.log("Logging...");
  console.log(constructor); //
}

@Logger // returns definition of the class because class is a syntactical sugar over constructor functions. Decorators execute when the class is defin
class Person {
  name = "Max";

  constructor() {
    console.log("Constructing person object...");
  }
}

const person = new Person();
console.log(person);
```

Working with Decorator Factories

1. Factor returns a decorator function, but we can configure it when we use it as a decorator to something.
2. Code:

```
function Logger(logString: string) {
  return function (constructor: Function) {
    console.log(logString);
    console.log(constructor); //
  };
}
```

```
@Logger("LOGGING - PERSON") // It is executed
class Person {
  name = "Max";

  constructor() {
    console.log("Constructing person object...");
  }
}

const person = new Person();
console.log(person);
```

Building More Useful Decorators

1. Code:

```
function Logger(logString: string) {
  return function (constructor: Function) {
    console.log(logString);
    console.log(constructor); //
  };
}

function WithTemplate(template: string, hookId: string) {
  return function (constructor: any) {
    const hookEl = document.getElementById(hookId);
    const p = new constructor();
    if (hookEl) {
      hookEl.innerHTML = template;
      hookEl.querySelector("h1")!.textContent = p.name;
    }
  };
}

// @Logger("LOGGING - PERSON") // It is executed
@WithTemplate("<h1>My Person Object</h1>", "app") // Can be exposed as part of a library
class Person {
  name = "Max";

  constructor() {
    console.log("Constructing person object...");
  }
}

const person = new Person();
console.log(person);
```

2. Code: Angular example

```
@Component({
  selector: "app-root",
  template: `
    <h1>{{ title }}</h1>
    <h2>My favorite hero is: {{ myHero }}</h2>
  `,
})
export class AppComponent {
  title = "Tour of Heroes";
  myHero = "Windstorm";
}
```

3. Decorators are utilities that other developers can use.

Adding Multiple Decorators

1. The decorator functions execute bottom up
2. The decorator factories run top down (like regular TS functions) (`@WithTemplate` and `@Logger`)
3. Code:

```
function Logger(logString: string) {
  return function (constructor: Function) {
    console.log(logString);
    console.log(constructor); //
  };
}

function WithTemplate(template: string, hookId: string) {
  return function (constructor: any) {
    console.log("Rendering template");
    const hookEl = document.getElementById(hookId);
    const p = new constructor();
    if (hookEl) {
      hookEl.innerHTML = template;
      hookEl.querySelector("h1")!.textContent = p.name;
    }
  };
}

@Logger("LOGGING - PERSON") // It is executed
@WithTemplate("<h1>My Person Object</h1>", "app") // Can be exposed as part of a library
class Person {
  name = "Max";

  constructor() {
    console.log("Constructing person object...");
  }
}

const person = new Person();
console.log(person);
```

Diving into Property Decorators

1. Where else can we add decorators?
2. Decorators on properties

```
class Product {
  @Log // It executes when class definition is registered by JS. When property is defined
  title: string;
  private _price: number;

  set price(value: number) {
    if (value > 0) {
      this._price = value;
    } else {
      throw new Error("Invalid price - should be positive");
    }
  }

  constructor(title: string, price: number) {
    this.title = title;
    this._price = price;
  }

  getPriceWithTax(tax: number) {
    return this._price * (1 + tax);
  }
}
```

Accessor & Parameter Decorators

1. Code:

```
// ---
function Log(target: any, propertyName: string | Symbol) {
  // For instance property - target = prototype
  // For static property - target = constructor function (?)
  console.log("Property decorator!");
  console.log(target, propertyName);
}

function Log2(target: any, name: string, descriptor: PropertyDescriptor) {
  console.log("Accessor decorator!");
  console.log(target); // prototype
  console.log(name); // `set` name
  console.log(descriptor); // setter function is defined, and getter not defined. It is configurable
}

function Log3(
  target: any,
  name: string | Symbol,
  descriptor: PropertyDescriptor
) {
  console.log("Method decorator!");
  console.log(target); // prototype
  console.log(name); // `set` name
  console.log(descriptor); // setter function is defined, and getter not defined. It is writable
}

function Log4(target: any, name: string | Symbol, position: number) {
  /*
  target - prototype
  name - name of the method in which we use the parameter
  position - of the argument (1st, ...)
  */
  console.log("Parameter decorator!");
  console.log(target); // prototype
  console.log(name); // `set` name
  console.log(position); // setter function is defined, and getter not defined. It is writable
}

class Product {
  @Log // It executes when class definition is registered by JS. When property is defined
  title: string;
  private _price: number;

  @Log2
  set price(value: number) {
    if (value > 0) {
      this._price = value;
    } else {
      throw new Error("Invalid price - should be positive");
    }
  }

  constructor(title: string, price: number) {
    this.title = title;
    this._price = price;
  }

  @Log3
  getPriceWithTax(@Log4 tax: number) {
    return this._price * (1 + tax);
  }
}
```

When Do Decorators Execute?

1. Code:

```
const p1 = new Product("Book", 19); // Decorators do not run when instance is created
const p2 = new Product("Book 2", 29); // Decorators do not run when instance is created
```

Returning (and changing) a Class in a Class Decorator

1. What a decorator returns depends on the type of decorator we are working with
2. `<T extends { new }>` - function that can be newed to generate a new object
3. Code: To extend and replace the old class

```
function WithTemplate(template: string, hookId: string) {
  return function <T extends { new (...args: any[]): { name: string } }>(
    originalConstructor: T
  ) {
    return class extends originalConstructor {
      constructor(..._: any[]) {
        // ensures template is rendered only when the class is instantiated
        super(); // calls originalConstructor
        console.log("Rendering template");
        const hookEl = document.getElementById(hookId);
        if (hookEl) {
          hookEl.innerHTML = template;
          hookEl.querySelector("h1")!.textContent = this.name;
        }
      }
    }; // new constructor function replaces original class/constructor function
  };
}

@Logger("LOGGING - PERSON") // It is executed
@WithTemplate("<h1>My Person Object</h1>", "app") // Can be exposed as part of a library
class Person {
  name = "Max";

  constructor() {
    console.log("Constructing person object...");
  }
}

const person = new Person();
console.log(person);
```

Other Decorator Return Types

1. We can return but not in all decorators
2. Other decorators which can return:
    1. Method decorators
    2. Accessor decorators (`get`, `set`)
3. TS will ignore return from decorators of properties and parameters
4. `PropertyDescriptor`
    1. `Object.defineProperty()`
        1. Defines a new property directly on an object, or modifies an existing property on an object, and returns the object
            1. Exmaple:

```
const object1 = {};

Object.defineProperty(object1, "property1", {
  value: 42,
  writable: false,
});

object1.property1 = 77;
// Throws an error in strict mode

console.log(object1.property1);
// Expected output: 42
```

    2. Properties
        1. `configurable: true`
            1. If we can change the config (delete property say)
        2. `enumerable: false`
            1. If it shows up when we loop through the object
                1. For method say
        3. `writable: true`
            1. Can we change the value after we initialize it
    3. One use-case:
        1. We can define new `set` and `get` methods

Example: Constructing an "Autobind" Decorator

1. Code: Vanilla JS example:

```
class Printer {
  message = "This works!";

  showMessage() {
    console.log(this.message);
  }
}

const p = new Printer();
```

```
const button = document.querySelector("button")!;
// button?.addEventListener("click", p.showMessage); // `this` refers to the target of the event. `addEventListener` binds `this` to the target of the
button?.addEventListener("click", p.showMessage.bind(p));
```

3. Code: `@Autobind`

```
function Autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
  const originalMethod = descriptor.value;
  const adjDescriptor: PropertyDescriptor = {
    configurable: true,
    enumerable: false,
    get() {
      // This method is executed when user tries to access this property. It is like `value` but with extra logic
      const boundFn = originalMethod.bind(this); // refers to the obj triggering the `get` method. It is the object on which we defined the getter.
      return boundFn;
    },
  };
  return adjDescriptor;
}

class Printer {
  message = "This works!";

  @Autobind
  showMessage() {
    console.log(this.message);
  }
}

const p = new Printer();

const button = document.querySelector("button")!;
// button?.addEventListener("click", p.showMessage); // `this` refers to the target of the event. `addEventListener` binds `this` to the target of the
button?.addEventListener("click", p.showMessage);
```

Validation with Decorators - First Steps

1. Code: Skeleton

```
function Required() {}

function PositiveNumber() {}

function validate(obj: object) {}

class Course {
  @Required
  title: string;

  @PositiveNumber
  price: number;

  constructor(t: string, p: number) {
    this.title = t;
    this.price = p;
  }
}

const courseForm = document.querySelector("form")!;
courseForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const titleEl = document.getElementById("title") as HTMLInputElement;
  const priceEl = document.getElementById("price") as HTMLInputElement;

  const title = titleEl.value;
  const price = +priceEl.value;

  if (!validate(createdCourse)) {
    alert("Invalid input, please try again!");
  }

  const createdCourse = new Course(title, price);
  console.log(createdCourse);
});
```

Validation with Decorators - Finished

1. Code:

```
interface ValidatorConfig {
  [property: string]: {
    [validatableProp: string]: string[]; // ['required', 'positive', ...]
  };
}

const registeredValidators: ValidatorConfig = {};

function Required(target: any, propName: string) {
  registeredValidators[target.constructor.name] = {
    ...registeredValidators[target.constructor.name],
    [propName]: ["required"],
  }; // exists in JS
}

function PositiveNumber(target: any, propName: string) {
  registeredValidators[target.constructor.name] = {
    ...registeredValidators[target.constructor.name],
    [propName]: ["positive"],
  }; // exists in JS
}
```

```
function validate(obj: any) {
  const objValidatorConfig = registeredValidators[obj.constructor.name];
  if (!objValidatorConfig) {
    return true;
  }
  let isValid = true;
  for (const prop in objValidatorConfig) {
    for (const validator of objValidatorConfig[prop]) {
      switch (validator) {
        case "required":
          isValid = isValid && !!obj[prop];
          break;
        case "positive":
          isValid = isValid && obj[prop] > 0;
          break;
      }
    }
  }
  return isValid;
}

class Course {
  @Required
  title: string;

  @PositiveNumber
  price: number;

  constructor(t: string, p: number) {
    this.title = t;
    this.price = p;
  }
}

const courseForm = document.querySelector("form")!;
courseForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const titleEl = document.getElementById("title") as HTMLInputElement;
  const priceEl = document.getElementById("price") as HTMLInputElement;

  const title = titleEl.value;
  const price = +priceEl.value;

  const createdCourse = new Course(title, price);

  if (!validate(createdCourse)) {
    alert("Invalid input, please try again!");
  } else {
    console.log(createdCourse);
  }
});
```

Fixing a Validator Bug

1. Code:

```
const registeredValidators: ValidatorConfig = {};

function Required(target: any, propName: string) {
  registeredValidators[target.constructor.name] = {
    ...registeredValidators[target.constructor.name],
    [propName]: [
      ...(registeredValidators[target.constructor.name]?.[propName] ?? []),
      "required",
    ],
  };
}

function PositiveNumber(target: any, propName: string) {
  registeredValidators[target.constructor.name] = {
    ...registeredValidators[target.constructor.name],
    [propName]: [
      ...(registeredValidators[target.constructor.name]?.[propName] ?? []),
      "positive",
    ],
  };
}
```

2. More refactoring
3. `??` - Returns its right-hand side operand when its left-hand side operand is `null` or `undefined`

Wrap Up

1. Search: TS Class Validator
    1. More elaborate implementation
    2. Can be imported to any project
2. Angular
    1. `@Component`
        1. To attach dynamic HTML code
3. NestJS
    1. Serverside JS framework for Node.js
    2. Heavily utilizes TS
        1. Embraces decorators
            1. `@Controller`
            2. `@Get()`
            3. ...

Useful Resources & Links

1. More on Decorators

Section 9: Practice Time! Let's Build a Drag & Drop Project

Module Introduction

1. Good to try next steps on my own

Getting Started

2. Download resources and copy paste the contents of `index.html` and `app.css`

DOM Element Selection & OOP Rendering

1. Code:

```
class ProjectInput {
  templateElement: HTMLTemplateElement;
  hostElement: HTMLDivElement;
  element: HTMLFormElement;

  constructor() {
    this.templateElement = document.getElementById(
      "project-input"
    )! as HTMLTemplateElement;
    this.hostElement = document.getElementById("app")! as HTMLDivElement;

    const importedNode = document.importNode(
      this.templateElement.content,
      true
    ); // `DocumentFragment`
    this.element = importedNode.firstElementChild as HTMLFormElement;
    this.attach();
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}

const prjInput = new ProjectInput();
```

Interacting with DOM Elements

1. Code:

```
class ProjectInput {
  templateElement: HTMLTemplateElement;
  hostElement: HTMLDivElement;
  element: HTMLFormElement;
  titleInputElement: HTMLInputElement;
  descriptionInputElement: HTMLInputElement;
  peopleInputElement: HTMLInputElement;

  constructor() {
    this.templateElement = document.getElementById(
      "project-input"
    )! as HTMLTemplateElement;
    this.hostElement = document.getElementById("app")! as HTMLDivElement;

    const importedNode = document.importNode(
      this.templateElement.content,
      true
    ); // `DocumentFragment`
    this.element = importedNode.firstElementChild as HTMLFormElement;
    this.element.id = "user-input";

    this.titleInputElement = this.element.querySelector(
      "#title"
    )! as HTMLInputElement;
    this.descriptionInputElement = this.element.querySelector(
      "#description"
    )! as HTMLInputElement;
    this.peopleInputElement = this.element.querySelector(
      "#people"
    )! as HTMLInputElement;

    this.configure();
    this.attach();
  }

  private submitHandler(event: Event) {
    event.preventDefault();
    console.log(this.titleInputElement.value);
  }

  private configure() {
    this.element.addEventListener("submit", this.submitHandler.bind(this));
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}
```

```
const prjInput = new ProjectInput();
```

Constructing & Using an "Autobind" Decorator

1. Code:

```
// autobind decorator
function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
  return {
    configurable: true,
    enumerable: false,
    get() {
      return descriptor.value.bind(this);
    },
  };
}

// ProjectInput class
class ProjectInput {
  templateElement: HTMLTemplateElement;
  hostElement: HTMLDivElement;
  element: HTMLFormElement;
  titleInputElement: HTMLInputElement;
  descriptionInputElement: HTMLInputElement;
  peopleInputElement: HTMLInputElement;

  constructor() {
    this.templateElement = document.getElementById(
      "project-input"
    )! as HTMLTemplateElement;
    this.hostElement = document.getElementById("app")! as HTMLDivElement;

    const importedNode = document.importNode(
      this.templateElement.content,
      true
    ); // `DocumentFragment`
    this.element = importedNode.firstElementChild as HTMLFormElement;
    this.element.id = "user-input";

    this.titleInputElement = this.element.querySelector(
      "#title"
    )! as HTMLInputElement;
    this.descriptionInputElement = this.element.querySelector(
      "#description"
    )! as HTMLInputElement;
    this.peopleInputElement = this.element.querySelector(
      "#people"
    )! as HTMLInputElement;

    this.configure();
    this.attach();
  }

  @autobind
  private submitHandler(event: Event) {
    event.preventDefault();
    console.log(this.titleInputElement.value);
  }

  private configure() {
    this.element.addEventListener("submit", this.submitHandler);
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}

const prjInput = new ProjectInput();
```

Fetching User Input

1. Code:

```
// Validation
interface Validatable {
  value: string | number;
  required?: boolean; // boolean | undefined
  minLength?: number;
  maxLength?: number;
  min?: number;
  max?: number;
}

function validate(validatableInput: Validatable) {
  let isValid = true;
  if (validatableInput.required) {
    isValid = isValid && validatableInput.value.toString().trim().length !== 0;
  }
  if (
    validatableInput.minLength != null &&
    typeof validatableInput.value === "string"
  ) {
    // `!= null` includes null and undefined
    isValid =
      isValid &&
      validatableInput.value.trim().length >= validatableInput.minLength;
  }
  if (
    validatableInput.maxLength != null &&
    typeof validatableInput.value === "string"
```

```
    ) {
      isValid =
        isValid &&
        validatableInput.value.trim().length <= validatableInput.maxLength;
    }
    if (
      validatableInput.min != null &&
      typeof validatableInput.value === "number"
    ) {
      isValid = isValid && validatableInput.value >= validatableInput.min;
    }
    if (
      validatableInput.max != null &&
      typeof validatableInput.value === "number"
    ) {
      isValid = isValid && validatableInput.value <= validatableInput.max;
    }
    return isValid;
  }

  // autobind decorator
  function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
    return {
      configurable: true,
      enumerable: false,
      get() {
        return descriptor.value.bind(this);
      },
    };
  }

  // ProjectInput class
  class ProjectInput {
    templateElement: HTMLTemplateElement;
    hostElement: HTMLDivElement;
    element: HTMLFormElement;
    titleInputElement: HTMLInputElement;
    descriptionInputElement: HTMLInputElement;
    peopleInputElement: HTMLInputElement;

    constructor() {
      this.templateElement = document.getElementById(
        "project-input"
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById("app")! as HTMLDivElement;

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLFormElement;
      this.element.id = "user-input";

      this.titleInputElement = this.element.querySelector(
        "#title"
      )! as HTMLInputElement;
      this.descriptionInputElement = this.element.querySelector(
        "#description"
      )! as HTMLInputElement;
      this.peopleInputElement = this.element.querySelector(
        "#people"
      )! as HTMLInputElement;

      this.configure();
      this.attach();
    }

    private gatherUserInput(): [string, string, number] | void {
      const enteredTitle = this.titleInputElement.value;
      const enteredDescription = this.descriptionInputElement.value;
      const enteredPeople = this.peopleInputElement.value;

      const titleValidatable: Validatable = {
        value: enteredTitle,
        required: true,
      };

      const descriptionValidatable: Validatable = {
        value: enteredDescription,
        required: true,
        minLength: 5,
      };

      const peopleValidatable: Validatable = {
        value: enteredPeople,
        required: true,
        min: 1,
        max: 5,
      };

      if (
        validate(titleValidatable) &&
        validate(descriptionValidatable) &&
        validate(peopleValidatable)
      ) {
        alert("Invalid input, please try again.");
        return;
      } else {
        return [enteredTitle, enteredDescription, +enteredPeople];
      }
    }

    private clearInputs() {
      this.titleInputElement.value = "";
```

```
    this.descriptionInputElement.value = "";
    this.peopleInputElement.value = "";
  }

  @autobind
  private submitHandler(event: Event) {
    event.preventDefault();
    console.log(this.titleInputElement.value);
    const userInput = this.gatherUserInput();

    if (Array.isArray(userInput)) {
      const [title, description, people] = userInput;
      console.log(title, description, people);
      this.clearInputs();
    }
  }

  private configure() {
    this.element.addEventListener("submit", this.submitHandler);
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}

const prjInput = new ProjectInput();
```

Constructing a Re-Usable Validation Functionality

Rendering Project Lists

```
// Validation
interface Validatable {
  value: string | number;
  required?: boolean; // boolean | undefined
  minLength?: number;
  maxLength?: number;
  min?: number;
  max?: number;
}

function validate(validatableInput: Validatable) {
  let isValid = true;
  if (validatableInput.required) {
    isValid = isValid && validatableInput.value.toString().trim().length !== 0;
  }
  if (
    validatableInput.minLength != null &&
    typeof validatableInput.value === "string"
  ) {
    // `!= null` includes null and undefined
    isValid =
      isValid &&
      validatableInput.value.trim().length >= validatableInput.minLength;
  }
  if (
    validatableInput.maxLength != null &&
    typeof validatableInput.value === "string"
  ) {
    isValid =
      isValid &&
      validatableInput.value.trim().length <= validatableInput.maxLength;
  }
  if (
    validatableInput.min != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value >= validatableInput.min;
  }
  if (
    validatableInput.max != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value <= validatableInput.max;
  }
  return isValid;
}

// autobind decorator
function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
  return {
    configurable: true,
    enumerable: false,
    get() {
      return descriptor.value.bind(this);
    },
  };
}

// ProjectList Class
class ProjectList {
  templateElement: HTMLTemplateElement;
  hostElement: HTMLDivElement;
  element: HTMLElement;

  constructor(private type: "active" | "finished") {
    this.templateElement = document.getElementById(
      "project-list"
    )! as HTMLTemplateElement;
    this.hostElement = document.getElementById("app")! as HTMLDivElement;

    const importedNode = document.importNode(
```

```
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLElement;
      this.element.id = `${this.type}-projects`;
      this.attach();
      this.renderContent();
    }

    private renderContent() {
      const listId = `${this.type}-projects-list`;
      this.element.querySelector("ul")!.id = listId;
      this.element.querySelector("h2")!.textContent =
        this.type.toUpperCase() + " PROJECTS";
    }

    private attach() {
      this.hostElement.insertAdjacentElement("beforeend", this.element);
    }
  }

  // ProjectInput class
  class ProjectInput {
    templateElement: HTMLTemplateElement;
    hostElement: HTMLDivElement;
    element: HTMLFormElement;
    titleInputElement: HTMLInputElement;
    descriptionInputElement: HTMLInputElement;
    peopleInputElement: HTMLInputElement;

    constructor() {
      this.templateElement = document.getElementById(
        "project-input"
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById("app")! as HTMLDivElement;

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLFormElement;
      this.element.id = "user-input";

      this.titleInputElement = this.element.querySelector(
        "#title"
      )! as HTMLInputElement;
      this.descriptionInputElement = this.element.querySelector(
        "#description"
      )! as HTMLInputElement;
      this.peopleInputElement = this.element.querySelector(
        "#people"
      )! as HTMLInputElement;

      this.configure();
      this.attach();
    }

    private gatherUserInput(): [string, string, number] | void {
      const enteredTitle = this.titleInputElement.value;
      const enteredDescription = this.descriptionInputElement.value;
      const enteredPeople = this.peopleInputElement.value;

      const titleValidatable: Validatable = {
        value: enteredTitle,
        required: true,
      };

      const descriptionValidatable: Validatable = {
        value: enteredDescription,
        required: true,
        minLength: 5,
      };

      const peopleValidatable: Validatable = {
        value: enteredPeople,
        required: true,
        min: 1,
        max: 5,
      };

      if (
        validate(titleValidatable) &&
        validate(descriptionValidatable) &&
        validate(peopleValidatable)
      ) {
        alert("Invalid input, please try again.");
        return;
      } else {
        return [enteredTitle, enteredDescription, +enteredPeople];
      }
    }

    private clearInputs() {
      this.titleInputElement.value = "";
      this.descriptionInputElement.value = "";
      this.peopleInputElement.value = "";
    }

    @autobind
    private submitHandler(event: Event) {
      event.preventDefault();
      console.log(this.titleInputElement.value);
      const userInput = this.gatherUserInput();
```

```
    if (Array.isArray(userInput)) {
      const [title, description, people] = userInput;
      console.log(title, description, people);
      this.clearInputs();
    }
  }

  private configure() {
    this.element.addEventListener("submit", this.submitHandler);
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}

const prjInput = new ProjectInput();
const activePrjList = new ProjectList("active");
const finishedPrjList = new ProjectList("finished");
```

Managing Application State with Singletons

1. Code:

```
// Project State Management
class ProjectState {
  private listeners: Function[] = [];
  private projects: any[] = [];
  private static instance: ProjectState;

  private constructor() {}

  static getInstance() {
    if (this.instance) {
      return this.instance;
    }
    this.instance = new ProjectState();
    return this.instance;
  }

  addListener(listenerFn: Function) {
    this.listeners.push(listenerFn);
  }

  addProject(title: string, description: string, numOfPeople: number) {
    const newProject = {
      id: Math.random().toString(),
      title,
      description,
      people: numOfPeople,
    };
    this.projects.push(newProject);
    for (const listenerFn of this.listeners) {
      listenerFn(this.projects.slice());
    }
  }
}

const projectState = ProjectState.getInstance();

// Validation
interface Validatable {
  value: string | number;
  required?: boolean; // boolean | undefined
  minLength?: number;
  maxLength?: number;
  min?: number;
  max?: number;
}

function validate(validatableInput: Validatable) {
  let isValid = true;
  if (validatableInput.required) {
    isValid = isValid && validatableInput.value.toString().trim().length !== 0;
  }
  if (
    validatableInput.minLength != null &&
    typeof validatableInput.value === "string"
  ) {
    // `!= null` includes null and undefined
    isValid =
      isValid &&
      validatableInput.value.trim().length >= validatableInput.minLength;
  }
  if (
    validatableInput.maxLength != null &&
    typeof validatableInput.value === "string"
  ) {
    isValid =
      isValid &&
      validatableInput.value.trim().length <= validatableInput.maxLength;
  }
  if (
    validatableInput.min != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value >= validatableInput.min;
  }
  if (
    validatableInput.max != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value <= validatableInput.max;
```

```
    }
    return isValid;
  }

  // autobind decorator
  function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
    return {
      configurable: true,
      enumerable: false,
      get() {
        return descriptor.value.bind(this);
      },
    };
  }

  // ProjectList Class
  class ProjectList {
    templateElement: HTMLTemplateElement;
    hostElement: HTMLDivElement;
    element: HTMLElement;
    assignedProjects: any[];

    constructor(private type: "active" | "finished") {
      this.templateElement = document.getElementById(
        "project-list"
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById("app")! as HTMLDivElement;
      this.assignedProjects = [];

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLElement;
      this.element.id = `${this.type}-projects`;

      projectState.addListener((projects: any[]) => {
        this.assignedProjects = projects;
        this.renderProjects();
      });

      this.attach();
      this.renderContent();
    }

    private renderProjects() {
      const listEl = document.getElementById(
        `${this.type}-projects-list`
      )! as HTMLUListElement;
      for (const prjItem of this.assignedProjects) {
        const listItem = document.createElement("li");
        listItem.textContent = prjItem.title;
        listEl?.appendChild(listItem);
      }
    }

    private renderContent() {
      const listId = `${this.type}-projects-list`;
      this.element.querySelector("ul")!.id = listId;
      this.element.querySelector("h2")!.textContent =
        this.type.toUpperCase() + " PROJECTS";
    }

    private attach() {
      this.hostElement.insertAdjacentElement("beforeend", this.element);
    }
  }

  // ProjectInput class
  class ProjectInput {
    templateElement: HTMLTemplateElement;
    hostElement: HTMLDivElement;
    element: HTMLFormElement;
    titleInputElement: HTMLInputElement;
    descriptionInputElement: HTMLInputElement;
    peopleInputElement: HTMLInputElement;

    constructor() {
      this.templateElement = document.getElementById(
        "project-input"
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById("app")! as HTMLDivElement;

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLFormElement;
      this.element.id = "user-input";

      this.titleInputElement = this.element.querySelector(
        "#title"
      )! as HTMLInputElement;
      this.descriptionInputElement = this.element.querySelector(
        "#description"
      )! as HTMLInputElement;
      this.peopleInputElement = this.element.querySelector(
        "#people"
      )! as HTMLInputElement;

      this.configure();
      this.attach();
    }
```

```
    private gatherUserInput(): [string, string, number] | void {
      const enteredTitle = this.titleInputElement.value;
      const enteredDescription = this.descriptionInputElement.value;
      const enteredPeople = this.peopleInputElement.value;

      const titleValidatable: Validatable = {
        value: enteredTitle,
        required: true,
      };

      const descriptionValidatable: Validatable = {
        value: enteredDescription,
        required: true,
        minLength: 5,
      };

      const peopleValidatable: Validatable = {
        value: enteredPeople,
        required: true,
        min: 1,
        max: 5,
      };

      if (
        !validate(titleValidatable) ||
        !validate(descriptionValidatable) ||
        !validate(peopleValidatable)
      ) {
        alert("Invalid input, please try again.");
        return;
      } else {
        return [enteredTitle, enteredDescription, +enteredPeople];
      }
    }

    private clearInputs() {
      this.titleInputElement.value = "";
      this.descriptionInputElement.value = "";
      this.peopleInputElement.value = "";
    }

    @autobind
    private submitHandler(event: Event) {
      event.preventDefault();
      console.log(this.titleInputElement.value);
      const userInput = this.gatherUserInput();

      if (Array.isArray(userInput)) {
        const [title, description, people] = userInput;
        // console.log(title, description, people);
        projectState.addProject(title, description, people);
        this.clearInputs();
      }
    }

    private configure() {
      this.element.addEventListener("submit", this.submitHandler);
    }

    private attach() {
      this.hostElement.insertAdjacentElement("afterbegin", this.element);
    }
  }

  const prjInput = new ProjectInput();
  const activePrjList = new ProjectList("active");
  const finishedPrjList = new ProjectList("finished");
```

More Classes & Custom Types

1. Code:

```
  // Project Type
  enum ProjectStatus {
    Active,
    Finished,
  }

  class Project {
    constructor(
      public id: string,
      public title: string,
      public description: string,
      public people: number,
      public status: ProjectStatus
    ) {}
  }

  // Project State Management
  type Listener = (items: Project[]) => void;

  class ProjectState {
    private listeners: Listener[] = [];
    private projects: Project[] = [];
    private static instance: ProjectState;

    private constructor() {}

    static getInstance() {
      if (this.instance) {
        return this.instance;
      }
      this.instance = new ProjectState();
```

```
    return this.instance;
  }

  addListener(listenerFn: Listener) {
    this.listeners.push(listenerFn);
  }

  addProject(title: string, description: string, numOfPeople: number) {
    const newProject = new Project(
      Math.random().toString(),
      title,
      description,
      numOfPeople,
      ProjectStatus.Active
    );
    this.projects.push(newProject);
    for (const listenerFn of this.listeners) {
      listenerFn(this.projects.slice());
    }
  }
}

const projectState = ProjectState.getInstance();

// Validation
interface Validatable {
  value: string | number;
  required?: boolean; // boolean | undefined
  minLength?: number;
  maxLength?: number;
  min?: number;
  max?: number;
}

function validate(validatableInput: Validatable) {
  let isValid = true;
  if (validatableInput.required) {
    isValid = isValid && validatableInput.value.toString().trim().length !== 0;
  }
  if (
    validatableInput.minLength != null &&
    typeof validatableInput.value === "string"
  ) {
    // `!= null` includes null and undefined
    isValid =
      isValid &&
      validatableInput.value.trim().length >= validatableInput.minLength;
  }
  if (
    validatableInput.maxLength != null &&
    typeof validatableInput.value === "string"
  ) {
    isValid =
      isValid &&
      validatableInput.value.trim().length <= validatableInput.maxLength;
  }
  if (
    validatableInput.min != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value >= validatableInput.min;
  }
  if (
    validatableInput.max != null &&
    typeof validatableInput.value === "number"
  ) {
    isValid = isValid && validatableInput.value <= validatableInput.max;
  }
  return isValid;
}

// autobind decorator
function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
  return {
    configurable: true,
    enumerable: false,
    get() {
      return descriptor.value.bind(this);
    },
  };
}

// ProjectList Class
class ProjectList {
  templateElement: HTMLTemplateElement;
  hostElement: HTMLDivElement;
  element: HTMLElement;
  assignedProjects: Project[];

  constructor(private type: "active" | "finished") {
    this.templateElement = document.getElementById(
      "project-list"
    )! as HTMLTemplateElement;
    this.hostElement = document.getElementById("app")! as HTMLDivElement;
    this.assignedProjects = [];

    const importedNode = document.importNode(
      this.templateElement.content,
      true
    ); // `DocumentFragment`
    this.element = importedNode.firstElementChild as HTMLElement;
    this.element.id = `${this.type}-projects`;

    projectState.addListener((projects: Project[]) => {
```

```
        this.assignedProjects = projects;
        this.renderProjects();
      });

      this.attach();
      this.renderContent();
    }

    private renderProjects() {
      const listEl = document.getElementById(
        `${this.type}-projects-list`
      )! as HTMLUListElement;
      for (const prjItem of this.assignedProjects) {
        const listItem = document.createElement("li");
        listItem.textContent = prjItem.title;
        listEl?.appendChild(listItem);
      }
    }

    private renderContent() {
      const listId = `${this.type}-projects-list`;
      this.element.querySelector("ul")!.id = listId;
      this.element.querySelector("h2")!.textContent =
        this.type.toUpperCase() + " PROJECTS";
    }

    private attach() {
      this.hostElement.insertAdjacentElement("beforeend", this.element);
    }
  }

  // ProjectInput class
  class ProjectInput {
    templateElement: HTMLTemplateElement;
    hostElement: HTMLDivElement;
    element: HTMLFormElement;
    titleInputElement: HTMLInputElement;
    descriptionInputElement: HTMLInputElement;
    peopleInputElement: HTMLInputElement;

    constructor() {
      this.templateElement = document.getElementById(
        "project-input"
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById("app")! as HTMLDivElement;

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as HTMLFormElement;
      this.element.id = "user-input";

      this.titleInputElement = this.element.querySelector(
        "#title"
      )! as HTMLInputElement;
      this.descriptionInputElement = this.element.querySelector(
        "#description"
      )! as HTMLInputElement;
      this.peopleInputElement = this.element.querySelector(
        "#people"
      )! as HTMLInputElement;

      this.configure();
      this.attach();
    }

    private gatherUserInput(): [string, string, number] | void {
      const enteredTitle = this.titleInputElement.value;
      const enteredDescription = this.descriptionInputElement.value;
      const enteredPeople = this.peopleInputElement.value;

      const titleValidatable: Validatable = {
        value: enteredTitle,
        required: true,
      };

      const descriptionValidatable: Validatable = {
        value: enteredDescription,
        required: true,
        minLength: 5,
      };

      const peopleValidatable: Validatable = {
        value: enteredPeople,
        required: true,
        min: 1,
        max: 5,
      };

      if (
        !validate(titleValidatable) ||
        !validate(descriptionValidatable) ||
        !validate(peopleValidatable)
      ) {
        alert("Invalid input, please try again.");
        return;
      } else {
        return [enteredTitle, enteredDescription, +enteredPeople];
      }
    }

    private clearInputs() {
      this.titleInputElement.value = "";
```

```
    this.descriptionInputElement.value = "";
    this.peopleInputElement.value = "";
  }

  @autobind
  private submitHandler(event: Event) {
    event.preventDefault();
    console.log(this.titleInputElement.value);
    const userInput = this.gatherUserInput();

    if (Array.isArray(userInput)) {
      const [title, description, people] = userInput;
      // console.log(title, description, people);
      projectState.addProject(title, description, people);
      this.clearInputs();
    }
  }

  private configure() {
    this.element.addEventListener("submit", this.submitHandler);
  }

  private attach() {
    this.hostElement.insertAdjacentElement("afterbegin", this.element);
  }
}

const prjInput = new ProjectInput();
const activePrjList = new ProjectList("active");
const finishedPrjList = new ProjectList("finished");
```

Filtering Projects with Enums

Adding Inheritance & Generics

Rendering Project Items with a Class

Using a Getter

Utilizing Interfaces to Implement Drag & Drop

Drag Events & Reflecting the Current State in the UI

Adding a Droppable Area

Finishing Drag & Drop

Wrap Up

1. Code: Full code

```
// Drag & Drop Interfaces
interface Draggable {
  dragStartHandler(event: DragEvent): void;
  dragEndHandler(event: DragEvent): void;
}

interface DragTarget {
  dragOverHandler(event: DragEvent): void;
  dropHandler(event: DragEvent): void;
  dragLeaveHandler(event: DragEvent): void;
}

// Project Type
enum ProjectStatus {
  Active,
  Finished,
}

class Project {
  constructor(
    public id: string,
    public title: string,
    public description: string,
    public people: number,
    public status: ProjectStatus
  ) {}
}

// Project State Management
type Listener<T> = (items: T[]) => void;

class State<T> {
  protected listeners: Listener<T>[] = [];

  addListener(listenerFn: Listener<T>) {
    this.listeners.push(listenerFn);
  }
}

class ProjectState extends State<Project> {
  private projects: Project[] = [];
  private static instance: ProjectState;

  private constructor() {
    super();
  }
```

```
    static getInstance() {
      if (this.instance) {
        return this.instance;
      }
      this.instance = new ProjectState();
      return this.instance;
    }

    addProject(title: string, description: string, numOfPeople: number) {
      const newProject = new Project(
        Math.random().toString(),
        title,
        description,
        numOfPeople,
        ProjectStatus.Active
      );
      this.projects.push(newProject);
      this.updateListeners();
    }

    moveProject(projectId: string, newStatus: ProjectStatus) {
      const project = this.projects.find((prj) => prj.id === projectId);
      if (project && project.status !== newStatus) {
        project.status = newStatus;
        this.updateListeners();
      }
    }

    private updateListeners() {
      for (const listenerFn of this.listeners) {
        listenerFn(this.projects.slice());
      }
    }
  }

  const projectState = ProjectState.getInstance();

  // Validation
  interface Validatable {
    value: string | number;
    required?: boolean; // boolean | undefined
    minLength?: number;
    maxLength?: number;
    min?: number;
    max?: number;
  }

  function validate(validatableInput: Validatable) {
    let isValid = true;
    if (validatableInput.required) {
      isValid = isValid && validatableInput.value.toString().trim().length !== 0;
    }
    if (
      validatableInput.minLength != null &&
      typeof validatableInput.value === "string"
    ) {
      // `!= null` includes null and undefined
      isValid =
        isValid &&
        validatableInput.value.trim().length >= validatableInput.minLength;
    }
    if (
      validatableInput.maxLength != null &&
      typeof validatableInput.value === "string"
    ) {
      isValid =
        isValid &&
        validatableInput.value.trim().length <= validatableInput.maxLength;
    }
    if (
      validatableInput.min != null &&
      typeof validatableInput.value === "number"
    ) {
      isValid = isValid && validatableInput.value >= validatableInput.min;
    }
    if (
      validatableInput.max != null &&
      typeof validatableInput.value === "number"
    ) {
      isValid = isValid && validatableInput.value <= validatableInput.max;
    }
    return isValid;
  }

  // autobind decorator
  function autobind(_1: any, _2: string, descriptor: PropertyDescriptor) {
    return {
      configurable: true,
      enumerable: false,
      get() {
        return descriptor.value.bind(this);
      },
    };
  }

  // Component Base Class
  abstract class Component<T extends HTMLElement, U extends HTMLElement> {
    templateElement: HTMLTemplateElement;
    hostElement: T;
    element: U;

    constructor(
      templateId: string,
```

```
      hostElementId: string,
      insertAtStart: boolean,
      newElementId?: string
    ) {
      this.templateElement = document.getElementById(
        templateId
      )! as HTMLTemplateElement;
      this.hostElement = document.getElementById(hostElementId)! as T;

      const importedNode = document.importNode(
        this.templateElement.content,
        true
      ); // `DocumentFragment`
      this.element = importedNode.firstElementChild as U;

      if (newElementId) {
        this.element.id = newElementId;
      }

      this.attach(insertAtStart);
    }

    private attach(insertABeginning: boolean) {
      this.hostElement.insertAdjacentElement(
        insertABeginning ? "afterbegin" : "beforeend",
        this.element
      );
    }

    abstract configure?(): void;

    abstract renderContent(): void;
  }

  // ProjectItem Class
  class ProjectItem
    extends Component<HTMLUListElement, HTMLLIElement>
    implements Draggable
  {
    private project: Project;

    get persons() {
      if (this.project.people === 1) {
        return "1 person";
      } else {
        return `${this.project.people} persons`;
      }
    }

    constructor(hostId: string, project: Project) {
      super("single-project", hostId, false, project.id);
      this.project = project;
      this.configure();
      this.renderContent();
    }

    @autobind
    dragStartHandler(event: DragEvent): void {
      event.dataTransfer!.setData("text/plain", this.project.id);
      event.dataTransfer!.effectAllowed = "move";
    }

    @autobind
    dragEndHandler(_: DragEvent): void {
      console.log("DragEnd");
    }

    configure(): void {
      this.element.addEventListener("dragstart", this.dragStartHandler);
      this.element.addEventListener("dragend", this.dragEndHandler);
    }

    renderContent(): void {
      this.element.querySelector("h2")!.textContent = this.project.title;
      this.element.querySelector("h3")!.textContent = this.persons + " assigned";
      this.element.querySelector("p")!.textContent = this.project.description;
    }
  }

  // ProjectList Class
  class ProjectList
    extends Component<HTMLDivElement, HTMLElement>
    implements DragTarget
  {
    assignedProjects: Project[];

    constructor(private type: "active" | "finished") {
      super("project-list", "app", false, `${type}-projects`);
      this.assignedProjects = [];
      this.configure(); // called here because inherited class constructor might set some values that this method might depend on.
      this.renderContent();
    }

    @autobind
    dragOverHandler(event: DragEvent): void {
      if (event.dataTransfer && event.dataTransfer.types[0] == "text/plain") {
        event.preventDefault(); // otherwise, drop is allowed on an element, if in drag over handler `preventDefault()` (since default behavior is do no
        const listEl = this.element.querySelector("ul")!;
        listEl.classList.add("droppable");
      }
    }

    @autobind
    dropHandler(event: DragEvent): void {
```

```
      const prjId = event.dataTransfer!.getData("text/plain");
      projectState.moveProject(
        prjId,
        this.type === "active" ? ProjectStatus.Active : ProjectStatus.Finished
      );
    }

    @autobind
    dragLeaveHandler(_: DragEvent): void {
      const listEl = this.element.querySelector("ul")!;
      listEl.classList.remove("droppable");
    }

    configure(): void {
      this.element.addEventListener("dragover", this.dragOverHandler);
      this.element.addEventListener("dragleave", this.dragLeaveHandler);
      this.element.addEventListener("drop", this.dropHandler);

      projectState.addListener((projects: Project[]) => {
        const relevantProjects = projects.filter((prj) => {
          if (this.type === "active") {
            return prj.status === ProjectStatus.Active;
          }
          return prj.status === ProjectStatus.Finished;
        });
        this.assignedProjects = relevantProjects;
        this.renderProjects();
      });
    }

    renderContent() {
      const listId = `${this.type}-projects-list`;
      this.element.querySelector("ul")!.id = listId;
      this.element.querySelector("h2")!.textContent =
        this.type.toUpperCase() + " PROJECTS";
    }

    private renderProjects() {
      const listEl = document.getElementById(
        `${this.type}-projects-list`
      )! as HTMLUListElement;
      listEl.innerHTML = "";
      for (const prjItem of this.assignedProjects) {
        new ProjectItem(this.element.querySelector("ul")!.id, prjItem);
      }
    }
  }

  // ProjectInput class
  class ProjectInput extends Component<HTMLDivElement, HTMLFormElement> {
    titleInputElement: HTMLInputElement;
    descriptionInputElement: HTMLInputElement;
    peopleInputElement: HTMLInputElement;

    constructor() {
      super("project-input", "app", true, "user-input");

      this.titleInputElement = this.element.querySelector(
        "#title"
      )! as HTMLInputElement;
      this.descriptionInputElement = this.element.querySelector(
        "#description"
      )! as HTMLInputElement;
      this.peopleInputElement = this.element.querySelector(
        "#people"
      )! as HTMLInputElement;

      this.configure();
    }

    configure() {
      this.element.addEventListener("submit", this.submitHandler);
    }

    renderContent(): void {}

    private gatherUserInput(): [string, string, number] | void {
      const enteredTitle = this.titleInputElement.value;
      const enteredDescription = this.descriptionInputElement.value;
      const enteredPeople = this.peopleInputElement.value;

      const titleValidatable: Validatable = {
        value: enteredTitle,
        required: true,
      };

      const descriptionValidatable: Validatable = {
        value: enteredDescription,
        required: true,
        minLength: 5,
      };

      const peopleValidatable: Validatable = {
        value: enteredPeople,
        required: true,
        min: 1,
        max: 5,
      };

      if (
        !validate(titleValidatable) ||
        !validate(descriptionValidatable) ||
        !validate(peopleValidatable)
      ) {
```

```
      alert("Invalid input, please try again.");
      return;
    } else {
      return [enteredTitle, enteredDescription, +enteredPeople];
    }
  }

  private clearInputs() {
    this.titleInputElement.value = "";
    this.descriptionInputElement.value = "";
    this.peopleInputElement.value = "";
  }

  @autobind
  private submitHandler(event: Event) {
    event.preventDefault();
    console.log(this.titleInputElement.value);
    const userInput = this.gatherUserInput();

    if (Array.isArray(userInput)) {
      const [title, description, people] = userInput;
      // console.log(title, description, people);
      projectState.addProject(title, description, people);
      this.clearInputs();
    }
  }
}

const prjInput = new ProjectInput();
const activePrjList = new ProjectList("active");
const finishedPrjList = new ProjectList("finished");
```

Useful Resources & Links

1. More on Drag & Drop

Section 10: Modules & Namespaces

Module Introduction

1. Topics:
    1. Modular code
        1. Split code into multiple files and each file stays manageable and extendable
        2. We can export from and to.
        3. Connected by
            1. Browser
            2. TypeScript
            3. Third party build tool
    2. Module content
        1. Two Options to Organize Code in Multiple Files

Writing Module Code - Your Options

1. Splitting code into multiple files
    1. Namespaces & File Building
        1. Namespaces: Use "namespace" code syntax to group code
            1. We can import namespaces into other files
        2. Pre-file or bundled compilaions is possible (less imports to manage)
            1. TS bundles files into one file
    2. ES6 Imports/Exports (ES6 modules)
        1. Modern JS has solution to the problem (OOTB)
            1. Import and export statements - Modern browsers understand it
                1. TS also supports import/export syntax
    3. Per-file compilation but single `<script>` import
        1. Modern browsers how to fetch the dependencies
            1. We can bundle files together and ship single file for production (Webpack can be used for it)
        2. Bundling via third-party tools (e.g. Webpack) is possible!

Working with Namespaces

1. TS is told where to find the other files. However, the files are compiled to standalone JS files (the connection is destroyed).
2. Code:

```
/// <reference path="drag-drop-interfaces.ts" />
/// <reference path="project-model.ts" />

namespace App {
  export ...
}
```

Organizing Files & Folders

A Problem with Namespace Imports

Important: Use Chrome or Firefox

1. Next lecture we would use features that work only in modern browsers.
    1. Firefox and Chrome
2. Subsequently, making it work in older browsers

Using ES Modules

1. Browser supports ES6 modules + TS supports and gives type support
2. tsconfig.json

```
{
  "target": "es6", // or higher
  "module": "es2015"
}
```

Fix for `exports is not defined` issue https://www.youtube.com/watch?v=KRLszVu7mXY

1. Code: tsconfig.json

```
{
  "compilerOptions": {
    /* Basic Options */
    "target": "ESNext" /* Specify ECMAScript target version: 'ES3' (default), 'ES5', 'ES2015', 'ES2016', 'ES2017','ES2018' or 'ESNEXT'. */,
    "module": "ESNext" /* Specify module code generation: 'none', 'commonjs', 'amd', 'system', 'umd', 'es2015', or 'ESNext'. */,
    "lib": [
      "dom",
      "ESNext",
      "dom.iterable",
      "scripthost"
    ] /* Specify library files to be included in the compilation. */,
    // "allowJs": true,                       /* Allow javascript files to be compiled. */
    // "checkJs": true,                       /* Report errors in .js files. */
    // "jsx": "preserve",                     /* Specify JSX code generation: 'preserve', 'react-native', or 'react'. */
    // "declaration": true,                   /* Generates corresponding '.d.ts' file. */
    // "declarationMap": true,                /* Generates a sourcemap for each corresponding '.d.ts' file. */
    "sourceMap": true /* Generates corresponding '.map' file. */,
    // "outFile": "./dist/bundle.js" /* Concatenate and emit output to single file. */,
    "outDir": "./dist" /* Redirect output structure to the directory. */,
    "rootDir": "./src" /* Specify the root directory of input files. Use to control the output directory structure with --outDir. */,
    // "composite": true,                     /* Enable project compilation */
    "removeComments": true /* Do not emit comments to output. */,
    // "noEmit": true,                        /* Do not emit outputs. */
    // "importHelpers": true,                 /* Import emit helpers from 'tslib'. */
    // "downlevelIteration": true,            /* Provide full support for iterables in 'for-of', spread, and destructuring when targeting 'ES5' or 'ES
    // "isolatedModules": true,               /* Transpile each file as a separate module (similar to 'ts.transpileModule'). */
    "noEmitOnError": true,

    /* Strict Type-Checking Options */
    "strict": true /* Enable all strict type-checking options. */,
    // "noImplicitAny": false,                 /* Raise error on expressions and declarations with an implied 'any' type. */
    // "strictNullChecks": true,              /* Enable strict null checks. */
    // "strictFunctionTypes": true,           /* Enable strict checking of function types. */
    // "strictBindCallApply": true,           /* Enable strict 'bind', 'call', and 'apply' methods on functions. */
    // "strictPropertyInitialization": true,  /* Enable strict checking of property initialization in classes. */
    // "noImplicitThis": true,                /* Raise error on 'this' expressions with an implied 'any' type. */
    // "alwaysStrict": true,                  /* Parse in strict mode and emit "use strict" for each source file. */

    /* Additional Checks */
    "noUnusedLocals": true /* Report errors on unused locals. */,
    "noUnusedParameters": true /* Report errors on unused parameters. */,
    "noImplicitReturns": true /* Report error when not all code paths in function return a value. */,
    // "noFallthroughCasesInSwitch": true,    /* Report errors for fallthrough cases in switch statement. */

    /* Module Resolution Options */
    "moduleResolution": "node" /* Specify module resolution strategy: 'node' (Node.js) or 'classic' (TypeScript pre-1.6). */,
    // "baseUrl": "./",                       /* Base directory to resolve non-absolute module names. */
    // "paths": {},                           /* A series of entries which re-map imports to lookup locations relative to the 'baseUrl'. */
    // "rootDirs": [],                        /* List of root folders whose combined content represents the structure of the project at runtime. */
    // "typeRoots": [],                       /* List of folders to include type definitions from. */
    // "types": [],                           /* Type declaration files to be included in compilation. */
    // "allowSyntheticDefaultImports": true,  /* Allow default imports from modules with no default export. This does not affect code emit, just typec
    "esModuleInterop": true /* Enables emit interoperability between CommonJS and ES Modules via creation of namespace objects for all imports. Implie
    // "preserveSymlinks": true,              /* Do not resolve the real path of symlinks. */

    /* Source Map Options */
    // "sourceRoot": "",                      /* Specify the location where debugger should locate TypeScript files instead of source locations. */
    // "mapRoot": "",                         /* Specify the location where debugger should locate map files instead of generated locations. */
    // "inlineSourceMap": true,               /* Emit a single file with source maps instead of having a separate file. */
    // "inlineSources": true,                 /* Emit the source alongside the sourcemaps within a single file; requires '--inlineSourceMap' or '--sou

    /* Experimental Options */
    "experimentalDecorators": true /* Enables experimental support for ES7 decorators. */
    // "emitDecoratorMetadata": true,         /* Enables experimental support for emitting type metadata for decorators. */
  },
  "exclude": [
    "node_modules" // would be the default
  ]
}
```

2. Code: index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta http-equiv="X-UA-Compatible" content="ie=edge" />
    <title>ProjectManager</title>
    <link rel="stylesheet" href="app.css" />
    <script type="module" src="dist/app.js" defer></script>
  </head>

  <body>
    <template id="project-input">
      <!-- Not rendered immediately. Reached via TS or JS and then render it control via JS and TS -->
      <form>
```

```
        <div class="form-control">
          <label for="title">Title</label>
          <input type="text" id="title" />
        </div>
        <div class="form-control">
          <label for="description">Description</label>
          <textarea id="description" rows="3"></textarea>
        </div>
        <div class="form-control">
          <label for="people">People</label>
          <input type="number" id="people" step="1" min="0" max="10" />
        </div>
        <button type="submit">ADD PROJECT</button>
      </form>
    </template>
    <template id="single-project">
      <li draggable="true">
        <h2></h2>
        <h3></h3>
        <p></p>
      </li>
    </template>
    <template id="project-list">
      <section class="projects">
        <header>
          <h2></h2>
        </header>
        <ul></ul>
      </section>
    </template>
    <div id="app"></div>
  </body>
</html>
```

3. Code: app.ts

```
import { ProjectInput } from "./components/project-input.js";
import { ProjectList } from "./components/project-list.js";

const prjInput = new ProjectInput();
const activePrjList = new ProjectList("active");
const finishedPrjList = new ProjectList("finished");

console.log(prjInput, activePrjList, finishedPrjList);
```

Understanding various Import & Export Syntaxes

1. Bundling imports
    1. Multiple imports can be grouped

    ```
    import Component from "./base-component.js"; // default export. Any name can be used
    import * as Validation from "../util/validation.js";
    import { autobind as Autobind } from "../decorators/autobind.js";

    // ...

    if (
      !Validation.validate(titleValidatable) ||
      !Validation.validate(descriptionValidatable) ||
      !Validation.validate(peopleValidatable)
    ) {
      alert("Invalid input, please try again.");
      return;
    } else {
      return [enteredTitle, enteredDescription, +enteredPeople];
    }

    //...

    @Autobind
    private submitHandler(event: Event) {
     // ...
    }
    ```

2. Named exports:
    1. They can be used to enforce naming convension

How Does Code in Modules Execute?

1. An exported file gets executed when it is imported by the first file. If another file imports it, it doesn't run

Wrap Up

1. ES Modules
    1. Extra type safety
    2. Ensures that every file can clearly specify what it wants
        1. With namespaces, if one file imports something, the other files can use it
            1. Problem: If the file changes, the other files may not work.
                1. Recommended for smaller projects
                2. If cannot use ES modules or a bundler
                3. If cannot use modern browser
    3. Problems:
        1. No support in older browsers
            1. Solution: Using a bundler
                1. Only one file is generated

1. For multiple HTTP requests, there is additional work

Useful Resources & Links

1. JavaScript Modules
2. More on ES Modules

Section 11: Using Webpack with TypeScript

Module Introduction

1. Another disadvantages and solutions
2. Module content
    1. What is Webpack?
    2. Example Setup
        1. Best of all worlds
            1. No disadvantage
            2. Splitting into multiple modules

What is Webpack & Why do we need it?

1. webpack.js.org
    1. Documentation is good
2. What is Webpack?
    1. Webpack is a Bundling & "Build Orchestration" Tool
        1. Reduces HTTP requests by bundling code together
        2. Optimizes code & allows more build steps (example: CSS modules etc.)
3. Normal setup:
    1. Multiple .ts files & imports (http requests) - not great
    2. Unoptimized code (not as small as possible)
        1. To shorten variable and function names (tools can do this automatically)
            1. Less download, less startup time
    3. "External" development server needed
        1. Tools can spinup server and reload with changes
4. With Webpack
    1. Code bundles, less imports required
    2. Optimized (minified) code, less code to download
    3. More build steps can be added easily (Webpack docs)

Installing Webpack & Important Dependencies

1. Installation:

```
npm i --save-dev webpack webpack-cli webpack-dev-server typescript ts-loader
```

```
1. `webpack` - heart of setup
    1. Allows to plugin certain functionalities to bundle the code & transform the code (transform & bundle)
2. `webpack-cli` - to run webpack commands in the project
3. `webpack-dev-server` - starts webpack under the hood and automatically monitors for changes and triggers recompile and servers the page
4. `typescript` - goot practice to install a copy per project
    1. Maintains a specific typescript version
        1. Global typescript dependency is eliminated
5. `ts-loader` - works with Webpack
    1. Tells how to convert TS code to JS code (Webpack can do compile with the help of ts-loader - it use tsc under the hood)
    2. Webpack then bundles emitted JS files into one bundle file
```

Adding Entry & Output Configuration

1. tsconfig.json
    1. `target` - `es6` or `es5`
    2. `module` - `es2015` or `es6`
    3. `outDir` - `./dist`
    4. `rootDir` - not required
        1. Webpack determines where the files are
2. webpack.config.js - tells how to work with the project
    1. Uses JS and Node.js features
    2. It exports a JS object
    3. We need to specify a sharting file (transitive dependencies are automatically resolved)
    4. We need to remove `.js` extension from imports or Webpack to work properly
        1. Webpack looks for `.js` extension
3. Config:

```
const path = require("path"); // core Node.js module

module.exports = {
  entry: "./src/app.ts",
  output: {
    // filename: "bundle.[contenthash].js", // every build generates a unique hash. It helps with caching in browser
    filename: "bundle.js",
    path: path.resolve(__dirname, "dist"), // It should match tsconfig.json to avoid errors. It builds absolute path
  },
};
```

Adding TypeScript Support with the ts-loader Package

1. webpack.config.js

```
const path = require("path"); // core Node.js module
```

```
module.exports = {
  entry: "./src/app.ts",
  output: {
    // filename: "bundle.[contenthash].js", // every build generates a unique hash. It helps with caching in browser
    filename: "bundle.js",
    path: path.resolve(__dirname, "dist"), // It should match tsconfig.json to avoid errors. It builds absolute path
  },
  devtool: "inline-source-map", // tells Webpack that there will be source-maps already. It should extract and wire up correctly to the bundle it gene
  module: {
    // just a file. Multiple rules can be added for different tyes of files
    rules: [
      {
        test: /\.ts$/, // Webpack runs a test to check if the rule applies to the file or not. It is regexp. Ends with .ts
        use: "ts-loader", // what to do with the files. ts-loader uses tsconfig.json.
        exclude: /node_modules/, // Webpack will not look into this folder
      },
    ],
  },
  resolve: {
    // tells which extension it should to the files in the imports it finds
    extensions: [".ts", ".js"], // `.js` is default
  },
};
```

2. package.json

```
{
  "name": "understanding-typescript",
  "version": "1.0.0",
  "description": "Understanding TypeScript Course Setup",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "lite-server",
    "build": "webpack"
  },
  "keywords": ["typescript", "course"],
  "author": "Maximilian Schwarzmüller",
  "license": "ISC",
  "devDependencies": {
    "lite-server": "^2.5.4",
    "ts-loader": "^9.5.1",
    "typescript": "^5.6.3",
    "webpack": "^5.96.1",
    "webpack-cli": "^5.1.4",
    "webpack-dev-server": "^5.1.0"
  },
  "dependencies": {
    "@babel/plugin-transform-modules-commonjs": "^7.25.9",
    "common-js": "^0.3.8",
    "understanding-typescript": "file:"
  }
}
```

3. index.html

```
<head>
  <script type="module" src="dist/bundle.js" defer></script>
</head>
```

4. In Dev tools, we can debug original TS files directly (sourcemaps are used)

Adjust Webpack Config

1. For latest webpack version, edit the file as follows:

    1. Add `devServer` option

    ```
    devServer: {
      static: [
        {
          directory: path.join(__dirname),
        },
      ];
    }
    ```

    2. Set `output.publicPath` to `'/dist/'`

    ```
    output: {
      filename: 'bundle.js',
      path: path.resolve(__dirname, 'dist'),
      publicPath: '/dist/'
    }
    ```

    3. Finished webpack.config.js

    ```
    const path = require("path");

    module.exports = {
      mode: "development",
      entry: "./src/app.ts",
      devServer: {
        static: [
          {
            directory: path.join(__dirname),
          },
        ],
      },
      output: {
        filename: "bundle.js",
        path: path.resolve(__dirname, "dist"),
        publicPath: "/dist/",
    ```

```
        },
        module: {
          rules: [
            {
              test: /\.tsx?$/,
              use: "ts-loader",
              exclude: /node_modules/,
            },
          ],
        },
        resolve: {
          extensions: [".ts", ".js"],
        },
      };
```

Finishing the Setup & Adding webpack-dev-server

1. webpack.config.js

```
const path = require("path"); // core Node.js module

module.exports = {
  mode: "development", // tells webpack that this will be for development. Fewer optimizations. Meaningful error messages
  entry: "./src/app.ts",
  devServer: {
    static: [
      {
        directory: path.join(__dirname),
      },
    ],
  },
  output: {
    // filename: "bundle.[contenthash].js", // every build generates a unique hash. It helps with caching in browser
    filename: "bundle.js",
    path: path.resolve(__dirname, "dist"), // It should match tsconfig.json to avoid errors. It builds absolute path
    publicPath: "/dist/", // For webpack to understand where output is written (by default it expects in this folder where we run this script)
  },
  devtool: "inline-source-map", // tells Webpack that there will be source-maps already. It should extract and wire up correctly to the bundle it gene
  module: {
    // just a file. Multiple rules can be added for different tyes of files
    rules: [
      {
        test: /\.ts$/, // Webpack runs a test to check if the rule applies to the file or not. It is regexp. Ends with .ts
        use: "ts-loader", // what to do with the files. ts-loader uses tsconfig.json.
        exclude: /node_modules/, // Webpack will not look into this folder
      },
    ],
  },
  resolve: {
    // tells which extension it should to the files in the imports it finds
    extensions: [".ts", ".js"], // `.js` is default
  },
};
```

Adding a Production Workflow

1. Plugin:

```
npm i --save-dev clean-webpack-plugin # to clean up `dist` folder whenever we rebuild our project
```

2. webpack.config.prod.js

```
const path = require("path"); // core Node.js module
const { CleanPlugin } = require("webpack");
const { CleanWebpackPlugin } = require("clean-webpack-plugin");

module.exports = {
  mode: "production", // tells webpack to do optimizations (minifies code, ...).
  entry: "./src/app.ts",
  devServer: {
    static: [
      {
        directory: path.join(__dirname),
      },
    ],
  },
  output: {
    // filename: "bundle.[contenthash].js", // every build generates a unique hash. It helps with caching in browser
    filename: "bundle.js",
    path: path.resolve(__dirname, "dist"), // It should match tsconfig.json to avoid errors. It builds absolute path
  },
  // devtool: "none", // source-maps are not required in production
  module: {
    // just a file. Multiple rules can be added for different tyes of files
    rules: [
      // applied to per file level
      {
        test: /\.ts$/, // Webpack runs a test to check if the rule applies to the file or not. It is regexp. Ends with .ts
        use: "ts-loader", // what to do with the files. ts-loader uses tsconfig.json.
        exclude: /node_modules/, // Webpack will not look into this folder
      },
    ],
  },
  resolve: {
    // tells which extension it should to the files in the imports it finds
    extensions: [".ts", ".js"], // `.js` is default
  },
  plugins: [new CleanWebpackPlugin()], // plugins apply to the entire project, to the general workflow. Before writing to output folder, we clean ever
};
```

3. package.json

```
{
  "name": "understanding-typescript",
  "version": "1.0.0",
  "description": "Understanding TypeScript Course Setup",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "webpack-dev-server",
    "build": "webpack --config webpack.config.prod.js"
  },
  "keywords": ["typescript", "course"],
  "author": "Maximilian Schwarzmüller",
  "license": "ISC",
  "devDependencies": {
    "clean-webpack-plugin": "^4.0.0",
    "lite-server": "^2.5.4",
    "ts-loader": "^9.5.1",
    "typescript": "^5.6.3",
    "webpack": "^5.96.1",
    "webpack-cli": "^5.1.4",
    "webpack-dev-server": "^5.1.0"
  },
  "dependencies": {
    "@babel/plugin-transform-modules-commonjs": "^7.25.9",
    "common-js": "^0.3.8",
    "understanding-typescript": "file:"
  }
}
```

Wrap Up

Useful Resources & Links

1. Official Webpack Docs

Section 12: 3rd Party Libraries & TypeScript

Module Introduction

1. Third party libraries
    1. How to add them to TS projects
2. Module content:
    1. "Normal" libraries & using them with TypeScript
        1. Can be used in regular JS projects
        2. Use them to get the most out of the TS
    2. TypeScript-specific libraries
        1. The libraries utilize certain type of features to give a way of working with them.

Using JavaScript (!) Libraries with TypeScript

1. Utility library: Lodash
    1. Example of any JS library
    2. Installation: `npm i --save lodash`
2. TS doesn't understand pure JS libraries. (however, we can generate final output with error and JS library works)
    1. Solution: Translating to TS
        1. Install `types` for third-party library
            1. `@types/lodash`
                1. `npm i --save @types/lodash`
                    1. Github repository: DefiniteyTyped - translations of 3rd party libraries. `.d.ts` (search for `jquery types`)
                        1. They contain instructions to TS (how something works and what's included) - they define types (we can write d.ts files)
                            1. Errors are gone.
                            2. Autocomplete is enabled

Using "declare" as a "Last Resort"

1. What if we don't have `types` package?
2. What if we have other JS code where we set global variable which needs to be re-used?

```
<script>
  const GLOBAL = "THIS IS SET";
</script>
```

```
import _ from "lodash"; // TS doesn't understand it because `lodash` is written in JS.

console.log(_.shuffle([1, 2, 3]));

declare var GLOBAL: string; // to declare features or variables. For the ones developer knows will be there

console.log(GLOBAL);
```

No Types Needed: class-transformer

1. `class-transformer`, `class-validator`
2. Traditional method of transformation:

```
const products = [
  { title: "A Carpet", price: 29.99 },
  { title: "A Book", price: 10.99 },
]; // We get from server say.

const loadedProducts = products.map(
  (prod) => new Product(prod.title, prod.price)
);
```

```
for (const prod of loadedProducts) {
  console.log(prod.getInformation());
}
```

3. Using `class-tranformer` - converts data into right models
    1. `npm i class-transformer --save`
    2. `npm i reflect-metadata --save` (dependency)
    3. `import "reflect-metadata";` - in root entry file (`App.ts`)
    4. It is developed from TS but also works for JS (because JS has `class`)
4. Code: app.ts

```
import _ from "lodash"; // TS doesn't understand it because `lodash` is written in JS.
import { Product } from "./product.model";
import "reflect-metadata";
import { plainToInstance } from "class-transformer";

console.log(_.shuffle([1, 2, 3]));

declare var GLOBAL: string;

console.log(GLOBAL);

const p1 = new Product("A Book", 12.99);
console.log(p1.getInformation());

const products = [
  { title: "A Carpet", price: 29.99 },
  { title: "A Book", price: 10.99 },
]; // We get from server say.

// const loadedProducts = products.map(
//   (prod) => new Product(prod.title, prod.price)
// );

const loadedProducts = plainToInstance(Product, products);

for (const prod of loadedProducts) {
  console.log(prod.getInformation());
}
```

TypeScript-embrading: class-validator

1. `class-validator` - uses TS. Builds on decorators. Gives a new way to work with them.
    1. We don't have to build decorators. They are provided.
    2. It helps us add validation with decorators in class
    3. Installation: `npm i class-validator --save`
2. app.ts

```
import _ from "lodash"; // TS doesn't understand it because `lodash` is written in JS.
import { Product } from "./product.model";
import "reflect-metadata";
import { plainToInstance } from "class-transformer";
import { validate } from "class-validator";

console.log(_.shuffle([1, 2, 3]));

declare var GLOBAL: string;

console.log(GLOBAL);

const p1 = new Product("A Book", 12.99);
console.log(p1.getInformation());

const products = [
  { title: "A Carpet", price: 29.99 },
  { title: "A Book", price: 10.99 },
]; // We get from server say.

// const loadedProducts = products.map(
//   (prod) => new Product(prod.title, prod.price)
// );

const loadedProducts = plainToInstance(Product, products);

for (const prod of loadedProducts) {
  console.log(prod.getInformation());
}

const newProd = new Product("", -5.99);
validate(newProd).then((errors) => {
  if (errors.length > 0) {
    console.log("VALIDATION ERRORS!");
    console.log(errors);
  } else {
    console.log(newProd);
  }
});
```

3. produc.model.ts

```
import { IsNotEmpty, IsNumber, IsPositive } from "class-validator";

export class Product {
  @IsNotEmpty()
  title: string;

  @IsNumber()
  @IsPositive()
  price: number;
```

```
    constructor(t: string, p: number) {
      this.title = t;
      this.price = p;
    }

    getInformation() {
      return [this.title, `$${this.price}`];
    }
  }
```

Wrap Up

1. With TS, we can use JS packages with no issues (with workarounds like types)
2. `class-tranformer` - works with TS and JS
3. `class-validator` - works only for TS

Useful Resources & Links

Section 13: Time to Practice! Let's build a "Select & Share a Place" App (incl. Google Maps)

Module Introduction

1. App using 3rd-party libraries
    1. Send HTTP request
    2. Render map using Google maps

Project Setup

1. Extract from resources
2. Enter address, convert to coordinates, render coordinates on a map rendered using Google maps

Getting User Input

Setting Up a Google API Key

1. Google Maps Platform
    1. Get started
        1. Maps (google JS SDK)
        2. Places (geocoding API)
            1. Project
    2. Search: Google Geocoding API

Using Axios to Fetch Coordinates for an Entered Address

1. `fetch` - built into browsers
    1. Error handling can be a bit clunky
2. `axios` - 3rd party package to send HTTP requests
    1. `npm i --save axios`
3. Geocoding API URL: `https://maps.googleapis.com/maps/api/geocode/json?address=1600+Amphitheatre+Parkway,+Mountain+View,+CA&key=YOUR_API_KEY`

Rendering a Map with Google Maps (incl. Types!)

1. Code: index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta http-equiv="X-UA-Compatible" content="ie=edge" />
    <title>Understanding TypeScript</title>
    <link rel="stylesheet" href="app.css" />
  </head>

  <body>
    <div id="map">
      <p>Please enter an address!</p>
    </div>
    <form>
      <input type="text" id="address" />
      <button type="submit">SEARCH ADDRESS</button>
    </form>
    <script
      src="https://maps.googleapis.com/maps/api/js?key=AIzaSyAwQ1ESiilsONDYF0pp6ED8JTVAkw6a9gU&libraries=maps,marker&v=beta"
      defer
    ></script>
    <!-- prettier-ignore -->
    <script>(g => { var h, a, k, p = "The Google Maps JavaScript API", c = "google", l = "importLibrary", q = "__ib__", m = document, b = window; b =
      ({ key: "AIzaSyAwQ1ESiilsONDYF0pp6ED8JTVAkw6a9gU", v: "weekly" });</script>
    <script src="dist/bundle.js" defer></script>
  </body>
</html>
```

```
import axios from "axios";

const form = document.querySelector("form")!;
const addressInput = document.getElementById("address")! as HTMLInputElement;

const GOOGLE_API_KEY = "AIzaSyAwQ1ESiilsONDYF0pp6ED8JTVAkw6a9gU";

declare var google: any;
```

```
function searchAddressHandler(event: Event) {
  event.preventDefault();
  const enteredAddress = addressInput.value;

  // send this to Google's API!
  type GoogleGeocodingResponse = {
    results: {
      geometry: {
        location: {
          lat: number;
          lng: number;
        };
      };
    }[];
    status: "OK" | "ZERO_RESULTS";
  };
  axios
    .get<GoogleGeocodingResponse>(
      `https://maps.googleapis.com/maps/api/geocode/json?address=${encodeURI(
        enteredAddress
      )}&key=${GOOGLE_API_KEY}`
    )
    .then((response) => {
      if (response.data.status !== "OK") {
        throw new Error("Could not fetch location!");
      }
      const coordinates = response.data.results[0].geometry.location;
      console.log(coordinates);

      // Initialize and add the map
      let map;
      async function initMap(): Promise<void> {
        // Request needed libraries.
        //@ts-ignore
        const { Map } = await google.maps.importLibrary("maps");
        const { AdvancedMarkerElement } = await google.maps.importLibrary(
          "marker"
        );

        // The map, centered at Uluru
        map = new Map(document.getElementById("map") as HTMLElement, {
          zoom: 16,
          center: coordinates,
          mapId: "DEMO_MAP_ID",
        });

        // The marker, positioned at Uluru
        const marker = new AdvancedMarkerElement({
          map: map,
          position: coordinates,
          title: "Uluru",
        });

        console.log(marker);
      }

      const promise: Promise<void> = initMap();

      console.log(promise);
    })
    .catch((err) => console.log(err)); // Comes with TS translation package. Other libraries might need installing types
}

form?.addEventListener("submit", searchAddressHandler);
```

Working with Maps without a Credit Card

1. Openlayers - alternative to Google Maps

Useful Resources & Links

1. Google Maps Pricing
2. Google Geocoding API
3. Google Maps JS SDK

Section 14: React.js & TypeScript

Module Introduction

1. TS is not default for React

Setting Up a React + TypeScript Project

1. Setting up is difficult
    1. `create-react-app` - supports TS out of the box
        1. Documentation:
            1. Adding TS to existing project
            2. New project with TS
2. https://create-react-app.dev/docs/adding-typescript/
    1. `npm create-react-app my-app --template typescript`
3. `tsx` - For TS and JSX code

How Do React + TypeScript Work Together?

1. `npm start`
    1. Watches for changes and recompiles, builds React app, and serves it on live server
        1. `React.FC` - Functional Component (Longer version also exists: `React.FunctionComponent`)

Working with Props and Types for Props

Getting User Input with "refs"

Cross-Component Communication

Working with State & Types

Managing State Better

More Props & State Work

Adding Styling

Types for other React Features (e.g. Redux or Routing)

1. Redux:
    1. Documentation: Usage with TS
2. React Router
    1. `npm i --save react-router-dom`
    2. `npm i --save-dev @types/react-router-dom`
3. Code: components/NewTodo.css

```
import React, { useRef } from "react";
// import { Route } from "react-router-dom";

import "./NewTodo.css";

type NewTodoProps = {
  onAddTodo: (todoText: string) => void;
};

const NewTodo: React.FC<NewTodoProps> = (props) => {
  const textInputRef = useRef<HTMLInputElement>(null);

  const todoSubmitHandler = (event: React.FormEvent) => {
    event.preventDefault();
    const enteredText = textInputRef!.current!.value; // This will never be null
    // console.log(enteredText);
    props.onAddTodo(enteredText);
  };

  return (
    <form onSubmit={todoSubmitHandler}>
      <div className="form-control">
        <label htmlFor="todo-text">Todo Text</label>
        <input type="text" id="todo-text" ref={textInputRef} />
      </div>
      <button type="submit">ADD TODO</button>
    </form>
  );
};

export default NewTodo;
```

3. Code: TodoList.tsx

```
import React from "react";

import "./TodoList.css";

interface TodoListProps {
  items: { id: string; text: string }[];
  onDeleteTodo: (id: string) => void;
}

const TodoList: React.FC<TodoListProps> = (props) => {
  return (
    <ul>
      {props.items.map((todo) => (
        <li key={todo.id}>
          <span>{todo.text}</span>
          <button onClick={props.onDeleteTodo.bind(null, todo.id)}>
            DELETE
          </button>
        </li>
      ))}
    </ul>
  );
};

export default TodoList;
```

4. Code: App.tsx

```
import React, { useState } from "react";
import TodoList from "./components/TodoList";
import NewTodo from "./components/NewTodo";
import { Todo } from "./todo.model";
```

```
const App: React.FC = () => {
  const [todos, setTodos] = useState<Todo[]>([]);

  const todoAddHandler = (text: string) => {
    // console.log(text);
    setTodos((prevTodos) => [
      ...prevTodos,
      { id: Math.random().toString(), text: text },
    ]); // because ...todos may not return the latest updated todos
  };

  const todoDeleteHandler = (todoId: string) => {
    setTodos((prevTodos) => {
      return prevTodos.filter((todo) => todo.id !== todoId);
    });
  };

  return (
    <div className="App">
      <NewTodo onAddTodo={todoAddHandler} />
      <TodoList items={todos} onDeleteTodo={todoDeleteHandler} /> {/* items is fixed */}
    </div>
  );
};

export default App;
```

Wrap Up

Useful Resources & Links

1. Official React Docs
2. More React Resources
3. create-react-app + TypeScript Docs

Section 15: Node.js + Express & TypeScript

Module Introduction

Executing TypeScript Code with Node.js

Setting up a Project

Finished Setup & Working with Types (in Node + Express Apps)

Adding Middleware & Types

Working with Controllers & Parsing Request Bodies

More CRUD Operations

Wrap Up

Useful Resources & Links

Section 16: Course Roundup

Thanks for being part of the course!

1. Keep on learning
2. To get good at something
    1. Use it
    2. Apply
    3. Don't lose the knowledge without using the course material